

High-Performance RISC-V CSR Access in FPGAs: Optimized Microarchitecture for Efficient Decoding and Multiplexing

MIGUEL JIMÉNEZ ARRIBAS, AGUSTÍN MARTÍNEZ HELLÍN, and MANUEL PRIETO MATEO, Automática, University of Alcalá, Alcalá de Henares, Spain

Control and Status Registers (CSRs) are fundamental to the RISC-V architecture, providing a versatile interface for managing processor state, system configuration, and functionality like exception handling, debugging, and performance monitoring. However, their implementation in FPGA-based systems poses challenges due to the inherent constraints of FPGA resources. The need for atomic parallel access to all CSRs, coupled with limitations of conventional LUT-based multiplexing, increases logic depth and resource usage, degrading Fmax in large-scale implementations. This work explores these obstacles and introduces optimized mechanisms to improve CSR handling efficiency. A minimal microarchitectural environment was built to isolate and evaluate multiple access strategies, retaining only components essential to CSR interaction. Leveraging a heterogeneous design tailored to FPGA capabilities—featuring BRAM for decoding, DSPs for multiplexing, and LUTs, CARRYs and flip-flops to facilitate routing—the proposed CSR subsystem achieves performance enhancements ranging from 50% to over 300%, contingent upon configuration. The top-performing implementation reaches 250 MHz on Artix-7 FPGAs while simultaneously reducing area and dynamic power. These results challenge prevailing reliance on abstract electronic design, highlighting that hardware-aware low-level methodologies can yield superior quality-of-results (QoR) over pure behavioral descriptions. More broadly, the findings inform datapath optimization involving decoding and multiplexing in performance-critical and resource-constrained digital architectures.

CCS Concepts: • **Hardware** → **Reconfigurable logic and FPGAs; Datapath optimization; Logic synthesis; Technology-mapping; Physical design (EDA); Placement; Wire routing; Partitioning and floorplanning; Physical synthesis; Best practices for EDA; Simulation and emulation; Assertion checking; Static timing analysis**; • **Computer systems organization** → **Reduced instruction set computing**;

Additional Key Words and Phrases: FPGA, RISC-V, Control and Status Registers (CSR), decoding, multiplexing, performance optimization, digital design

ACM Reference format:

Miguel Jiménez Arribas, Agustín Martínez Hellín, and Manuel Prieto Mateo. 2026. High-Performance RISC-V CSR Access in FPGAs: Optimized Microarchitecture for Efficient Decoding and Multiplexing. *ACM Trans. Reconfig. Technol. Syst.* 19, 1, Article 5 (March 2026), 34 pages.
<https://doi.org/10.1145/3787491>

This work was supported by contract PRE2020-094740 under project “Energetic Particle Detector en Solar Orbiter: fase E, calibración y explotación de datos” reference: PID2019-104863RB-I00, funded by Spanish Ministerio de Ciencia, Innovación y Universidades (MCIN) (DOI: 10.13039/501100011033), Agencia Estatal de Investigación (AEI), and by the European Social Fund Plus (ESF+).

Authors’ Contact Information: Miguel Jiménez Arribas (corresponding author), Automática, University of Alcalá, Alcalá de Henares, Spain; e-mail: miguel.jimenez@uah.es; Agustín Martínez Hellín, Automática, University of Alcalá, Alcalá de Henares, Spain; e-mail: agustin.martinez@uah.es; Manuel Prieto Mateo, Automática, University of Alcalá, Alcalá de Henares, Spain; e-mail: manuel.prieto@uah.es.



This work is licensed under Creative Commons Attribution International 4.0.

© 2026 Copyright held by the owner/author(s).

ACM 1936-7414/2026/3-ART5

<https://doi.org/10.1145/3787491>

1 Introduction

Digital electronics is a rapidly evolving field, driven by the demand for high-performance and resource-efficient computing solutions across a wide range of applications. A cornerstone of this evolution is the proliferation of **Field-Programmable Gate Arrays (FPGAs)**, offering engineers versatile platforms for implementing sophisticated digital systems. In such systems, a frequent and valuable practice is the incorporation of softcore processors, that is, processors developed in reprogrammable fabric using hardware description languages. Softcores and FPGAs form a synergistic pairing; both emphasize flexibility and configurability, making them especially well-suited for implementing tailored computing architectures. In addition, their reprogrammability allows designers to employ a single hardware platform for multiple applications, rendering FPGAs especially cost-effective in prototyping and low-volume production, where fixed hardware and its associated non-recurring engineering expenses can be prohibitive [1]. They also facilitate iterative development, enabling rapid design refinement and optimization, which, along with their reusability, further diminishes development time and costs, solidifying their appeal across numerous domains.

These processors must adhere to a specific **Instruction Set Architecture (ISA)**. Among the various architectural options available, RISC-V, an open standard computer architecture, has gained notable popularity, with extensive adoption across academic, industrial, and hobbyist communities [2, 3]. Its appeal stems not only from its royalty-free, open source nature, and growing community support, but also from its high degree of customization. RISC-V features a modular design which enables engineers to implement only the subset of instructions necessary for a specific application, making it a synergistic characteristic for deployment on these programmable platforms. This flexibility is particularly valuable for space electronics, an area in which our research group specializes, where FPGA processors are favored for their adaptability, faster development life-cycle, and superior fault tolerance in radiation-prone environments via reconfiguration [4]. However, as the complexity of FPGA-based RISC-V systems increases, achieving optimal performance becomes a significant challenge, especially when resource limitations and power consumption are also considered. This work explores one such challenge in depth. Specifically, the article focuses on the implementation of the **Control and Status Registers (CSRs)**, where naive behavioral approaches can introduce significant performance bottlenecks as the number of registers grows.

The present study is conducted within the Space Research Group, a multidisciplinary team based at the University of Alcala, dedicated to the exploration of space sciences and technologies, bringing together scientists and engineers; including software and hardware designers, to advance research in these domains. In the context relevant to this work, building upon the experience in reliable on-board computing for missions like ESA's Solar Orbiter [5], the group holds extensive familiarity with FPGA-based systems, having integrated beamforming algorithms in such platforms [6], and operated with several soft processors, including LEON [7] and RISC-V [8]. In this latter architecture, our efforts to enhance processor observability with a synchronous **Hardware Performance Monitor (HPM)** capable of consistently supplying reliable data [9, 10] prompted the incorporation of a substantial number of CSRs. These are specialized registers responsible for managing and monitoring the privileged internal state of the processor.

It was during this incorporation that the performance impediment discussed in this work was identified. The requirement for frequent and parallel access to a considerable number of CSRs poses a challenge in FPGA implementations, preventing the use of RAM structures, which have a limited number of ports. Consequently, employing behavioral descriptions, synthesizers would recreate the CSR structure with **Flip-Flops (FFs)** for data storage, and look-up tables (LUTs), for address decoding and multiplexing. This causes a considerable number of long logical paths, which in turn increases resource utilization and power consumption.

The objective of this work is to analyze these underlying limitations and propose hardware-aware design strategies that managed to improve performance from 50% to over 300%, depending on the applied optimizations. The top-performing solution successfully achieves a single logic level across most of the design, operating at frequencies of up to 250 MHz, even on the lowest speed grade Artix-7 FPGA. This is accomplished through a heterogeneous approach that encourages cohesion of the design, by leveraging the capabilities of BRAMs for decoding and DSPs for multiplexing, in conjunction with LUTs, carry chains and FFs for switching and faster routing speed.

Regarding the broader applicability of the proposed design, this work was developed and assessed on a Digilent Nexys A7 development board [11], featuring an Artix-7 AMD Xilinx FPGA, one of the least demanding members of the 7-series family [12]. This platform was selected as a representative and conservative target to demonstrate the feasibility and efficiency of the microarchitecture under consideration. While the present work focuses on this device family, the architectural principles underlying the design remain applicable to other FPGA platforms. The expected portability and cross-vendor applicability of the proposed approach are further discussed in Section 5.

Therefore, although this work originated within a space-focused project, the findings extend beyond this realm, offering insights and practical solutions for engineers across numerous domains, encompassing academic researchers, industry professionals, and hobbyists. Nevertheless, given the inception of this project in space applications, it is important to contextualize the design practices commonly applied in this sector. In space electronics, the two-process behavioral description [13] has become a widely adopted methodology, largely due to its structured nature and its application in Jiri Gaisler's influential LEON processor. This technique separates combinational and sequential logic into distinct VHDL processes, enhancing readability and promoting portability across different FPGA platforms, a factor that contributes to the methodology's widespread use.

However, this level of abstraction is not universally advantageous. As this article illustrates, strict adherence to behavioral modeling can result in substantial inefficiencies in FPGA implementations, constituting this the article's central focus. These inefficiencies are particularly evident in comparing FPGA and **Application-Specific Integrated Circuit (ASIC)** implementations, as ASICs typically exhibit enhanced performance and power efficiency. Nonetheless, this gap has narrowed in recent years with integration of heterogeneous resources in modern FPGAs [14], as also evidenced in this study. While some considerations discussed here may extend to ASIC design, such as using RAM structures as efficient decoding mechanisms, the specific constraints and tradeoffs differ due to the increased structural flexibility inherent in ASICs, unrestricted from the fixed LUT architecture. Consequently, ASIC-specific aspects fall outside the scope of this study. Nonetheless, the broader principle remains effective hardware design demands careful alignment of the architecture with the implementation substrate.

This issue reflects a deeper trend in contemporary digital design practice, marked by increased emphasis on abstraction, precipitated by requirements of higher productivity and the availability of greater computational power. Frameworks for **High-Level Synthesis (HLS)** programming [15, 16] are a concrete manifestation of this abstraction-first approach. While abstraction is indispensable in managing complexity, it reduces opportunities for cultivating deep, system-level understanding, obscuring performance-critical aspects of FPGA implementations [16, 17]. Our intention is not to dismiss these modern tools; many applications benefit greatly from their use, but rather highlight the need to balance them with a rigorous understanding of the underlying hardware. In this spirit, our work serves as a counterpoint to the prevailing trend, illustrating that meticulous examination of the FPGA substrate, and targeted optimizations, can yield substantial improvements in performance, area, and power efficiency.

The remainder of this article is structured as follows. Section 2 defines the problem under consideration and surveys the relevant state of the art. Thereafter, in Section 3, the developments

undertaken in this work are discussed. Initially, an analysis of different methods for optimization in behavioral implementations is outlined, followed by an in-depth look at the design that demonstrates maximal performance; and accompanied by an exploration of ancillary factors pertinent to the efficacy of the design. Moving forward, Section 4 details the verification and validation strategies adopted to ensure functional correctness and integration readiness of the proposed design in the target processing environment. Subsequently, Section 5 shows an analytical comparison of the results, focusing on performance, area utilization, and power consumption of each of the implemented solutions. Finally, in Section 6, conclusions are drawn and potential future work is outlined.

2 Related Work

This section offers the required context to understand the rationale behind this work and its relation to existing solutions. Initially, the nature of CSRs is explored, alongside a definition of the problem under consideration, highlighting the specific obstacles the present study aims to address. Next, current solutions to multiport memory design are evaluated, with brief explanations of their underlying behavior and the reasons why these methods do not effectively address the unique requirements to aid CSR implementation for FPGA. By exploring these areas, a solid foundation is established to support comprehension of the enhancements proposed in the following sections.

2.1 Problem Definition: CSR Implementation in FPGA-Based RISC-V Processors

Within the RISC-V architecture, CSRs constitute the standardized interface between software and the internal control state of the processor. They govern configuration, privilege management, exception and interrupt handling, performance monitoring, and other system-level behaviors from the processor's operational context. The RISC-V privileged specification [18] organizes CSRs by privilege level, with the following order: debug (D), machine (M), hypervisor (H), supervisor (S), and user (U); each defining different scopes of visibility and control. Typical examples include `mstatus`, which manages the global enablement of traps and privilege mode transitions; or `mtvec` and `mepc`, which, respectively, hold the pointer to the trap vector, or the address of the last instruction executed prior to a trap, enabling precise exception and interruption recovery. In addition, performance-monitoring counters are also CSRs such as `mcycle` or `mhpmcounterX`, facilitating the profiling of diverse microarchitectural events.

CSRs are accessed through several dedicated instructions, namely `CSRRW` (for atomic read and write), and `CSRRS` and `CSRRC` (for atomic reading the full CSR, and then set or clear specific bits), alongside their immediate variants. The instructions implement atomic read-modify-write semantics to ensure coherent updates in concurrent or interrupt-prone contexts. These instructions are essential to guarantee precise control over exceptional behavior and to maintain coherence between hardware events and the software-visible state. From a hardware perspective, each access may involve conditional masked updates, privilege checks, and side effects, all of which can introduce datapath and timing complexity, especially when scaling to a larger subset of registers.

This challenge is exacerbated by the inherent structure of FPGA resources and the specialized nature of its hardware architecture, wherein the available resources are ultimately constrained by their predetermined purpose and fixed placement. This imposes limitations on how CSRs can be stored, multiplexed, and accessed in parallel. Naïve implementations often serialize accesses or rely on suboptimal behavioral implementations, as will be discussed throughout this document. This work analyzes these structural inefficiencies and proposes a hardware-aware CSR subsystem that leverages heterogeneous FPGA primitives to improve performance scalability, resource utilization and power consumption.

The main limitation is encountered when considering parallel access to the large number of CSRs stipulated by the RISC-V specification, which allows for up to 4,096 registers [18]. Due to their role in managing the processor's internal state, it is imperative that these registers can be accessed for both read and write operations concurrently, from many different regions of the processor. The central issue arises from multiplexing access to these registers, which, as mentioned, must be executed atomically; and thus typically, though not necessarily, within a single cycle [19]. To efficiently manage the large number of CSRs, a suitable selection mechanism must be employed.

In FPGA-based systems, access to large amounts of data generally involves the use of **Block RAM components (BRAMs)**, specialized structures engineered especially for this purpose, which handle the decoding of the address and the multiplexing of the output internally. However, they only allow sequential access to stored data, with maximum dual port access. Rather, the prevailing solution involves utilization of LUTs to manage decoding and multiplexing operations, and FFs to store the register values as this way multiple CSRs can be read and written at once. However, this approach introduces significant complexity by requiring multi-level logic to manage access paths to individual registers, increasing the demand on FPGA resources and potentially resulting in performance bottlenecks. In addition, the storage density achieved with individual FFs is considerably lower, leading to more sparse layouts. As the design scales, routing congestion becomes prominent, negatively impacting timing closure and system throughput by lengthening the critical path and reducing the maximum operating frequency. Moreover, the added complexity leads to higher resource utilization and therefore a heightened dynamic power consumption, due to greater switching activity, which can be particularly problematic for power-constrained applications.

In practice, the majority of implementations select a smaller subset, typically ranging from tens to several hundred registers, as explored in Section 2 of the online supplementary material. For the design presented in this article, a subset of 259 CSRs was adopted. The employed registers were selected to mimic potential functionality for a broad range of applications, thus enabling transferability of our findings to other contexts with minimal adaptation. This subset includes the CSRs required for exceptional flow control, which handle system exceptions, interrupts, and other critical control operations, such as privilege mode switching or extension discovery. In addition, since the work that originated this article already contained an HPM, the counters and their associated configuration registers were likewise incorporated. Furthermore, given that most high-performance processors require custom CSRs for augmented functionality, several of them were included to enhance the microarchitectures' applicability in diverse use cases. This approach ensures that the implementation remains versatile and directly transferable to typical FPGA-based RISC-V designs. However, even this reduced number of CSRs can still introduce significant overhead when attempting to address these registers in an FPGA.

Subsequently, a theoretical calculation will be undertaken to illustrate the complexity of the multiplexing process. Modern FPGA are predominantly based on 6-input LUTs, constraining the width and depth of programmed logic. Consequently, in the context of this work, each LUT can function as a 4:1 multiplexer, and thus, larger multiplexers must be constructed hierarchically using multiple stages. The depth of such mux tree for N inputs, denoted as $D(N)$, can be expressed as:

$$D(N) = \lceil \log_4(N) \rceil. \quad (1)$$

Therefore, for the implementations presented herein, with $N = 259$, the required logic depth is:

$$D(259) = \lceil \log_4(259) \rceil = \left\lceil \frac{\log_2 259}{\log_2 4} \right\rceil = \left\lceil \frac{8.02}{2} \right\rceil = 5. \quad (2)$$

Nevertheless, this calculation represents a lower bound for the required logic depth. In practice, the actual depth is typically greater, as showcased in Sections 3.1 and 5. The discrepancy arises due

to two main factors, the presence of supplementary logic beyond that required for multiplexing, and the sparsity of the implemented CSRs relative to the total available address space. Specifically, addressing all 4,096 potential registers requires 12 selection bits, whereas only 9 bits would suffice for the subset of 259 CSRs. Since the addresses are fixed by the RISC-V specification, this results in an uneven distribution of the multiplexer inputs, forcing the tree to become deeper than the idealized case. Additionally, it is reasonable to hypothesize that an even greater increase in the number of registers would further compound this complexity, exerting additional pressure on timing closure, and overall resource and power consumption. For instance, there are implementations with over 300 CSRs, as discussed in Section 2 of the supplementary material, which presents an examination of the impact of CSRs on the performance of several RISC-V processors in FPGA.

To address this challenge, it is necessary to explore alternative strategies by restructuring CSR access logic leveraging FPGA-specific architectural features. The following sections present a set of optimizations and introduce novel architectural solutions designed to minimize the performance penalty associated with CSR accesses. By improving access efficiency through hardware-level optimizations, such as the intelligent use of BRAMs and DSPs to eliminate traditional multiplexer trees, these solutions aim to maintain compliance with the RISC-V standard, while ensuring that CSR handling does not become a limiting factor in FPGA-based processor implementations.

2.2 State of the Art of Multiport Memory Solutions and Their Limitations: Multiplexing

In seeking architectural inspiration for more efficient CSR access mechanisms, the field of multiport memory design was examined. Efficient access to computational data are central to high-performance computing, particularly when multiple operands are fetched or updated within a single instruction cycle, as in superscalar architectures. Multiport memories enable such parallelism by allowing several concurrent read and write operations, and thus represent a potential avenue for mitigating the CSR access bottleneck observed in FPGA RISC-V processors. While these techniques were found not directly transferable to this work, they informed conception of the proposed microarchitecture. The following discussion reviews various existing FPGA multiport memory solutions, comparing their operational principles, strengths, and limitations with respect to scalability and resource efficiency.

Existing approaches can be grouped into three main categories: (i) register-based designs, (ii) physical or temporal memory replication, and (iii) algorithmic alternatives, including techniques such as: **Live Value Table (LVT)**, XOR-based architectures, and **Invalidation LVT (I-LVT)**.

Register-Based Memories. The first approach is labeled in the literature as register-based RAM or basic **Logic Elements (LE)** RAM [20, 21]. This method constructs multiport memories entirely from FPGA fabric resources using LUTs and FFs. As discussed in Section 2.1, this represents the default implementation, as no other memory primitive, such as block RAM or LUTRAM, can automatically match the behavioral description of the CSRs due to their lack of parallelism. Register-based designs achieve high operational frequencies for shallow memories, making them suitable for applications where memory size is limited. However, their tradeoff is scalability. As memory depth increases, each bit requires its own storage element and associated routing. This causes area and congestion to grow rapidly, resulting in higher power consumption and reduced routability.

Memory Replication and Time-Multiplexing. Conceptually, these strategies aim to generate the perception among external consumers that the memory possesses more ports than any individual primitive, either via physical replication or time multiplexing [20, 21]. Physical replication exists in two forms: full memory replication or bank replication. The first method can be employed for read access; however, it is not suitable for writing due to the inability to determine the location of the last written value. Instead, this is accomplished by dividing memory into multiple banks, each holding only a fraction of the total memory. The main limitation is that concurrent writes

may not evenly distribute across banks, requiring arbitration logic to manage conflicts, increasing delay and control logic. In addition, another drawback is that each port only uses its associated bank, preventing true simultaneous multiport access, which leads to underutilization of memory if access patterns fail to align with the banking structure, or if each bank is smaller than the size of the BRAMs.

In contrast to spatial replication, it is possible to time-multiplex memory, emulating availability of multiple ports via a process called multipumping. The memory is essentially overclocked, operating at a multiple of the system clock, allowing several consecutive accesses within one apparent cycle. This can be an effective method for increasing memory bandwidth without resource duplication, especially in moderate-speed designs. However, its success depends on the ability to close timing at frequencies several times higher than the processor clock, which is often unrealistic for dense FPGA systems targeting high clock rates. Consequently, both strategies become impractical for deep CSR implementations, where scalability and efficient BRAM utilization are paramount.

Algorithmic Memories. The overarching objective of these techniques is to create larger memories by employing RTL logic to consolidate several distinct primitives into a unified multiport memory [22, 23]. The concept was first established in the field of ASIC design and subsequently adapted to FPGA [20, 22]. Many alternatives exist, but this study focuses on the following three approaches.

To overcome the challenge of determining the location of a written value in replicated memories, Laforest et al. [20] proposed the LVT architecture. In this scheme, each write port has its own dedicated memory, and a separate, narrower table, the LVT, tracks the most recent value for each address. In the event of a write, the operation is directed to a particular replica, generally associated with the corresponding write port, and the LVT is updated to reflect which replica now holds the valid version of the modified address. Then, subsequent reads consult the LVT to select the correct replica. This decouples storage from write resolution, enabling true multiwrite operation. Notably, the LVT itself is a multiport RAM. Therefore, as the table holds an entry for every address, its depth equals that of the full memory. Hence, despite its narrow width, its implementation as register-based RAM can consume substantial logic and introduce long decoding paths. Furthermore, the replication of memory to support multiple read and write ports introduces a considerable area overhead, a phenomenon especially pronounced as the number of ports increases.

Another option outlined in the same work is the XOR-based method, which eliminates the explicit table exploiting the commutative and associative property of exclusive-OR operation: $A \oplus B \oplus B = A$. Similarly to the LVT design, each write port is assigned its own memory bank. However, in this case, when a write occurs, updates are performed by XORing the new data with the existing values retrieved from all the other banks. Then, upon a read operation, the design ensures the most recent value can be recovered by again XORing the contents of all banks. This simplifies the read path but imposes a two-cycle write sequence and demands many replicated BRAMs, especially as the number of ports grows. The number of BRAM memories employed by a XOR-based multiport design is calculated with the formula: $n_w \cdot ((n_w - 1) + n_r)$, where n_w and n_r are the number of write and read ports, respectively. Hence, for the design considered in this article, with 259 ports, this would result in 133903 BRAMs, which is unfeasible for any realistic implementation. Admittedly, this calculation concerns a general multiport memory, whereas CSRs would not require such generality, so the actual cost would be lower, but it still illustrates their scalability problem.

Lastly, the final examined methodology is the I-LVT technique [21]. This strategy implements the LVT using BRAM instead of basic fabric resources, encoding invalidation directly into the data path in an analogous form as the XOR-based feedback mechanism. This is accomplished via a carefully designed invalidation function, which first reads the data stored at the same address across all replicas and combines them with the new data. The resulting value is written to the

target bank, implicitly marking other replicas as invalid. During a subsequent read, an output extraction function processes the contents of all banks to recover the bank identifier from the most recent write to that address. Then, this value selects the data from the identified bank and multiplexes it, finally completing the read. This reduces logic usage, improving depth scalability compared to the LVT method. Still, this strategy exhibits several limitations. The development of an adequate invalidation logic is non-trivial, particularly for memories with elevated number of ports. Moreover, it presents the same drawbacks as the XOR-based methodology, namely the introduction of additional latency, and, most importantly, impractical BRAM consumption.

Multiplexing Overhead. A final aspect that warrants consideration is multiplexing. The primary benefit of the latter alternatives is found in the enhancement of the decoding path, facilitated by the integration of BRAMs. However, an additional limitation remains overhead when multiplexing the read output. All reviewed strategies are designed for use in modern processor multiport register files. As such, they assume modest port counts, overlooking the cost of multiplexing outputs at scale. Ultimately, the extraction and routing of the selected data to each read port must occur through multiplexing logic within the programmable fabric. As the system is scaled, this introduces additional logic and wiring, causing timing degradation and resource overhead. Xilinx documents [24, 25] examine the topic of implementing efficient, scalable multiplexers for FPGA in greater detail.

Among the strategies delineated therein, several conventional strategies are identified, including explicit instantiation of dedicated multiplexer primitives, like MUXF7 and MUXF8, to construct hierarchical trees that minimize interslice routing. Additionally, several more unconventional techniques are discussed. These include decomposition of the multiplexing operation into its constituent logic, i.e., AND gates to mask inputs based on selection lines, and a shared OR gate to combine the results. This provides an opportunity for various optimizations. For instance, if a one-hot selection scheme was considered, this logic could be embedded in FF reset or enable paths, eliminating the need for dedicated AND gates. Alternatively, underutilized components of the fabric, such as the MUXCY elements from the carry chain, can be repurposed as multiplexers. This approach offers a resource-efficient method for constructing priority-select and compact routing structures. These techniques not only reduce LUT usage but also alleviate routing congestion, which is often the dominant factor limiting the scalability and performance of large M:N multiplexers and, more broadly, of large-scale designs within the reprogrammable FPGA fabric. The benefit emerges from leveraging locality, thereby minimizing demands on the global interconnect fabric.

Summary. To conclude, existing FPGA multiport memory approaches each present a unique balance of latency, area, and timing tradeoffs, yet none scales efficiently to the extremely high port counts and routing demands of CSR subsystems. Table 1 summarizes their principal characteristics. Building upon these insights, the proposed design adopts a hybrid strategy, employing banked BRAMs to perform decoding efficiently, and avoiding duplication of resources by storing the data values themselves in a register-based RAM. Likewise, regarding multiplexing, the proposed solution incorporates several of the discussed concepts while simultaneously introducing a novel technique: repurposing DSP units as multiplexers, a strategy that, to the best of the authors' knowledge, has not been previously explored. These strategies yielded a compact design, enhancing routability, and improving overall performance. Further elaboration on the intricacies of this microarchitecture can be found in Section 3, with a subsequent demonstration of its efficacy reported in Section 5.

3 Development

This section delineates the methodological and architectural refinement progression underlying the development of the CSR subsystem for our RISC-V soft-processor targeting FPGA platforms. In order to better comprehend the contents of this section, it is important to first understand how the

Table 1. Comparison of Potential FPGA Multiport Memory Strategies for CSR Access Implementation

Strategy	Principle of operation	Advantages	Limitations
Register-based	Constructs memory using LUTs and FFs to emulate multiport behavior.	High frequency in shallow memories.	Poor scalability; rapid growth in area, routing congestion, and power.
Bank Replication	Physically duplicate memory to provide multiple read ports.	Less resource duplication than other alternatives; conceptually simple.	Still elevated area cost per port count; arbitration; potential under utilization.
Multipumping	Operate memory at a multiple of system clock to emulate extra ports.	Increases effective bandwidth with minimal resource duplication.	Stringent timing closure; unsuitable for high target frequencies or port counts.
LVT	Use an auxiliary table to track which bank holds the latest written value.	General multiport capability; good in shallow memories.	Poor performance scalability; logic and BRAM overhead; multiplexing.
XOR-based	Reconstruct most recent data using XOR of all memory banks.	Reduces logic utilization; simplifies read path avoiding selection logic.	Increased latency; scales to impractical BRAM consumption; multiplexing.
ILVT	Uses BRAM-based LVT and invalidation logic to reduce area.	Improves scalability of LVT; lower logic utilization.	Complex invalidation logic; poor BRAM scalability; multiplexing.

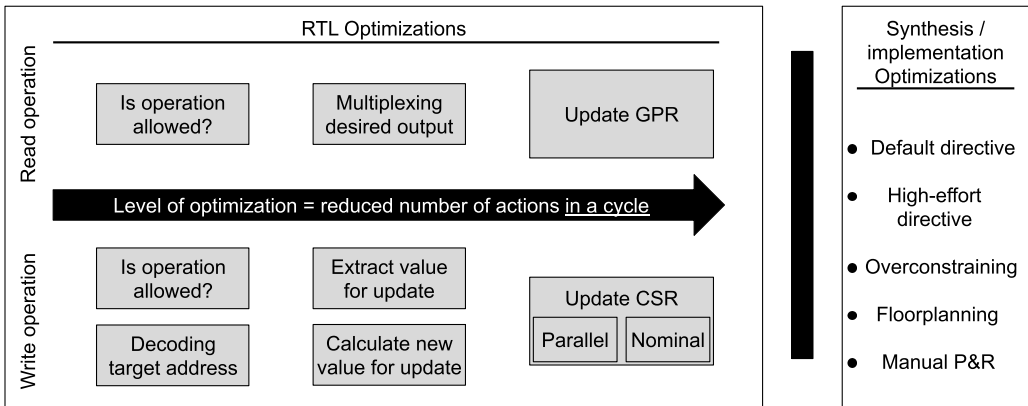


Fig. 1. Existing optimizations. The graph is divided into two sections, enumerating the potential optimizations examined throughout this work, both at the RTL level, and during the synthesis/implementation process. For RTL optimization, the objective is minimizing the number of actions required to achieve atomic CSR access, contingent upon the underlying hardware generated.

optimization process functioned. The main objective during this process was trying to minimize the amount of logic required to produce the atomic access of the CSRs, that is, to reduce the number of logical levels. To this end, it was imperative to limit the number of actions executed within a single cycle, as elucidated in Figure 1. However, this is not the sole consideration, as the best behavioral design, which exhibited the lowest number of actions, was nevertheless deemed unsatisfactory in terms of performance in comparison to the proposed hardware-aware alternative. This underscores the significance of incorporating the hardware substrate as the fundamental lever around which to orient development, as will be examined throughout this section. Following the optimization of the number of logical steps, the second objective was to minimize the time required for the data to traverse the die from one portion of the design to another. This was primarily achieved through the optimization of the place and route implementation.

The remainder of this section unfolds as follows. First, Section 3.1 outlines several optimization guidelines and practical considerations to improve CSR handling in high-level, purely behavioral implementations. As has been hypothesized in the preceding paragraphs and will be demonstrated in the Results Section, Section 5, although such descriptions facilitate flexibility, portability, and rapid development, they can also lead to performance degradation and increased resource and power consumption. This underscores the necessity for an alternative approach, which will be

elaborated in detail in Section 3.2. There, a refined CSR subsystem is introduced, founded on a more hardware-oriented methodology. When appropriate, the proposed architecture incorporates the explicit instantiation of low-level primitives, yielding significant improvements in performance and efficiency across nearly all evaluated metrics, as further substantiated in Section 5. Lastly, Section 3.3 briefly examines other ancillary factors, including implementation directives, placement constraints, and floorplanning techniques, which can also influence overall **Quality of Results (QoR)**.

3.1 Considerations on the Optimization of Behavioral Implementations

The process of attempting to integrate CSRs into an existing RISC-V architecture processor involves careful examination of numerous factors. Among these factors, atomicity emerges as a particularly salient consideration. The manner in which atomicity is implemented can have a significant impact not only on the CSR subsystem itself, but also on the broader processor microarchitecture and its components. As further explored in Section 2 of the supplementary material, the NEORV32 processor exemplifies a straightforward design. This processor utilizes a multi-cycle execution model allowing CSR instructions to span multiple cycles. This facilitates atomic operations through staged execution, as only when one instruction completes does the next instruction begin execution. However, in scenarios where the objective is to maximize performance, this multi-cycle approach is generally unsuitable. In such contexts, pipelined microarchitectures are preferable, as they support higher throughput of instructions. This shift necessitates an alternative strategy for handling CSR instructions, a challenge acknowledged by the authors of the RISC-V specification themselves [19].

A potential similar solution in pipelined architectures is to stall the execution of CSR instructions by controlling their entry into the pipeline. This ensures that when a CSR operation is identified, its initiation is suspended until the pipeline has been emptied, and then, the next instruction is not executed until the CSR operation has exited the processor. In a conventional five-stage pipeline, for instance, this approach enables the reliable execution of read operations during the ID stage, while the write operations could be performed during the WB stage. Although this approach is effective, it imposes limitations on throughput and results in underutilization of the available pipeline stages.

To mitigate this, an alternative strategy consists in restricting both CSR reads and writes to a single pipeline stage, typically to the WB or commit stage. This effectively allows multiple CSR instructions to coexist in the pipeline. This approach represents one of the techniques adopted by the behavioral implementations evaluated in Section 5. However, this strategy introduces a substantial challenge: it condenses all the operations that occurred in multiple stages in the previous method into a single cycle. In particular, first, the validity of the instruction must be ascertained. This is due to the possibility that a prior write could have altered the processor's state, rendering the current instruction illegal and triggering an exception. Second, the register to be accessed needs to be selected. The third step involves the calculation of the new value. In the event of a set/clear instruction, the prior value of the register must first be read. And lastly, subsequent to this, the read and write operations are finally performed. This solution suffers from the following drawbacks. The critical path is evidently extended and risks potentially becoming prohibitively large. In addition, the hardware of the ALU has to be duplicated in order to provide the functionality of setting and clearing specific parts of the registers. Nevertheless, modern FPGAs provide highly optimized carry chains that can mitigate this limitation to some extent, suggesting that such an approach may remain viable when performance is not the primary design constraint.

However, considering the merit of this approach, the design could be improved by incorporating certain optimizations. For instance, given the infrequent nature of CSR access in typical workloads, a control unit could be devised to stall execution as needed, allowing implementation of several other improvements, should the architecture be modified to accommodate them. Several refinements were

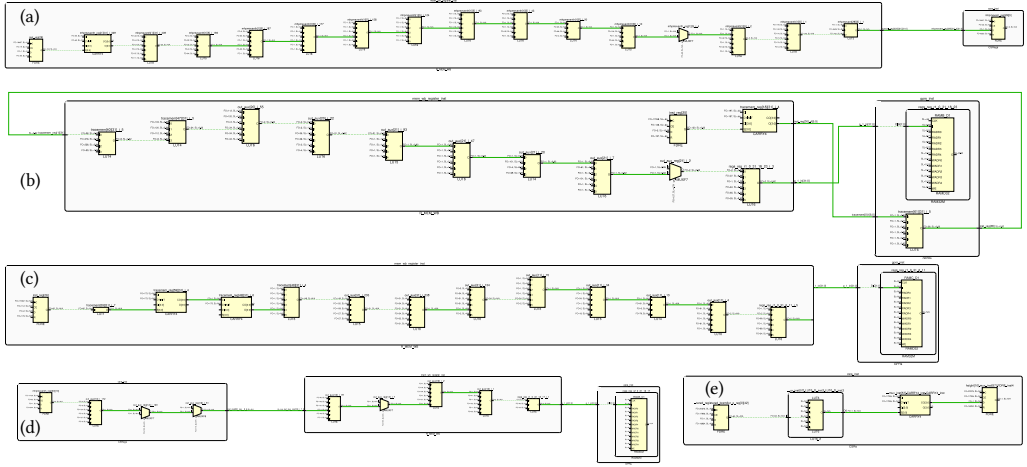


Fig. 2. Critical paths of implemented alternatives: (a) Baseline approach, with all operations in a single cycle (~15 logic levels); (b) Variant with set/clear operations in a separate pipeline stage (~12 levels); (c) Version employing (~8 levels) parallel exception calculation (~12 levels); (d) Optimized alternative after loop unrolling (~8 levels); and (e) Proposed subsystem (2 levels). This image is intended solely to illustrate the relative logic depth of each variant; the individual components within each path are not meant to be legible or analyzed in detail.

explored, as depicted in Figure 2, which highlights the impact of each configuration on the critical path within the pipeline. These are later analyzed in Section 5. The first improvement involves validating the CSR instructions in advance. If the microarchitecture can support early resolution of trap conditions, such as memory faults or privilege violations, then, CSR-related exceptions could similarly be detected during decoding, resulting in one less task performed at the WB stage.

Moreover, if the access control logic is further refined to anticipate data hazards among in-flight CSR instructions, it becomes possible to detect conflicts with pending CSR writes and stall the pipeline accordingly, avoiding the hardware duplication discussed previously. This is achieved by relocating the set/clear logic computation to the ALU, thereby removing a substantial portion of logic from this long combinational path. Consequently, the critical path would be solely limited to the decoding and multiplexing operations, which as explained in the initial sections, are intrinsic to behavioral descriptions and cannot be fully circumvented without more explicit hardware modeling. For a more detailed elaboration on the implementation of this approach, refer to Section 3.4 of [9].

Furthermore, a final behavioral optimization was identified through critical path analysis. Figure 2(c) illustrates that the longest combinational path still spans over 12 logic levels which exceeds the predicted depth based on the calculations from Section 2.1. This discrepancy was traced to the interaction between high-level VHDL constructs and synthesis heuristics. In particular, the use of loops within case statements was found to produce inefficient logic inference, likely due to the way the synthesis tools unroll or interpret such constructs. Upon restructuring this portion of the code to avoid iterative constructs in favor of explicitly enumerated logic branches, the resulting path depth aligned with expectations, especially when accounting for the sparsity of the address, as discussed in the aforementioned section. This finding once again confirms that coding style, even within high-level behavioral abstractions, can exert non-trivial effects on timing behavior, and highlights the critical role of synthesis-aware modeling practices in achieving optimal implementation results.

Finally, a potential additional optimization may be achieved through the implementation of a targeted reduction in width, utilizing solely the essential bits from each CSR. A common example of this is the `mstatus` CSR, that monitors the current operating state of the core. This register

manifests a broad range of functionalities, not all of which are implemented in every processor, and thus the fields associated with them can be omitted. By eliminating unused or unimplemented bits, FF usage can be reduced without compromising functional correctness. However, this does not reduce addressing logic complexity. Therefore, in scenarios involving processors with numerous CSRs that necessitate a full width, such as the statistical counters, the enhancement achieved through this optimization would be minimal. In this same context, an alternative optimization for these counters consists of constructing them with widths of 32 or smaller increments, as opposed to directly implementing the full 64 bits. These smaller slices would then be cascaded together to form the entire counter, effectively pipelining the counters, with the associated improvement in logic and reducing routing complexity through the device, albeit at the expense of increased latency. The efficacy of these approaches lies in their reduced timing and resource requirements in the FPGA. Nevertheless, it is worth noting that these optimizations could be applied in conjunction with all the other optimizations presented in this article, including the proposed optimized strategy discussed in the next subsection. Therefore, since by definition they would require *ad hoc* adaptation, these optimizations have not been implemented and are only mentioned here for completeness.

3.2 Proposed CSR Microarchitecture

This subsection details the proposed strategies designed to overcome the limitations of behavioral implementations, drawing inspiration from the relevant literature reviewed in Section 2.2. The presented CSR microarchitecture represents a deliberate shift toward a hardware-aware design paradigm. This approach emphasizes efficiency and careful resource utilization by aligning architectural decisions closely with the underlying FPGA fabric capabilities. To achieve maximum performance, certain critical components required explicit instantiation of low-level FPGA primitives. This selective instantiation was motivated by needing fine-grained control over implementation details that cannot always be guaranteed through automated synthesis of behavioral descriptions. Specifically, the design incorporates several heterogeneous blocks, leveraging the decoding capabilities of BRAM via a one-hot scheme, the routing abilities and data immediacy from FFs, the switching capabilities of LUTs, the strategic placement of CARRY modules, and the high width capacity of DSPs suitable for muxing. Collectively, by orchestrating these primitives with precise intent, the design improves timing closure, resource utilization, and power efficiency. In addition, this section outlines the conceptual framework and rationale underpinning the design choices of the proposed system, while Section 5 explores the applicability and QoR findings of the design.

As established in earlier sections, the primary concern with behavioral CSR implementations pertains to resource bottlenecks caused by register-based memories constructed from LE. When using LUTs for address decoding and multiplexing, these quickly become saturated as the number of CSRs increases. This saturation leads to widespread logic dispersion, with related logic placed far apart. This results in congestion and large routing delays, thereby making timing closure more difficult and, in general, causing inefficient utilization of FPGA resources. Meanwhile, other FPGA components remain underutilized, constituting an untapped potential for optimization that could enhance resource efficiency and boost performance. This forms the basis for the proposed design, employing a heterogeneous approach while simultaneously improving the cohesion of the system so that, if architected properly to avoid congestion, achieve significant speedup.

The first elements to be considered were BRAMs, since they are specifically engineered for the purpose of storing large amounts of data, hence the examination of multiport designs in Section 2.2. However, as discussed therein, none of the available techniques proved directly applicable to the subject of this work. Rather, the exploration of these strategies served as inspiration which is reflected in the final proposed approach. For instance, these strategies employed BRAMs to store the data to be accessed, but they exhibited two primary limitations. The LVT approach needed a

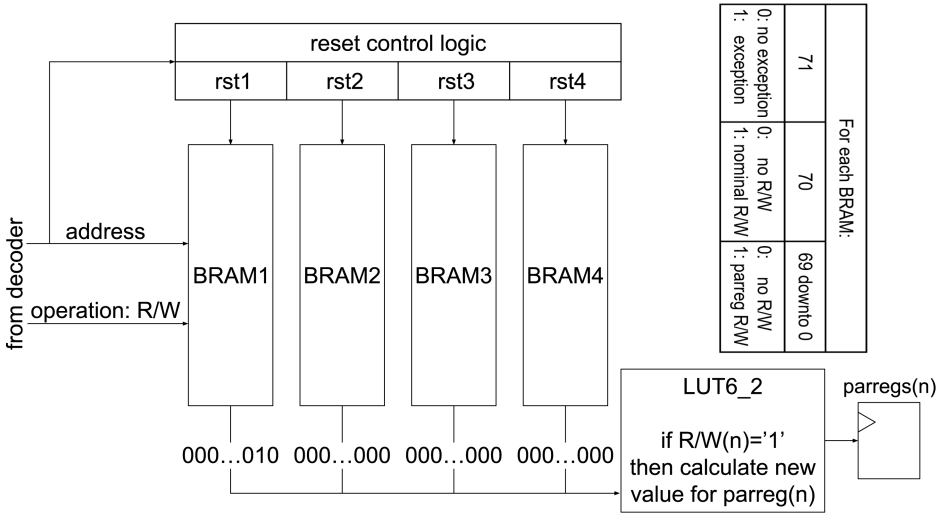


Fig. 3. Example of a BRAM-based one-hot decoding structure with banking. A single bit is activated in BRAM1, resulting in a write operation in the corresponding FF.

register-based RAM to track which BRAM had the latest copy of the data, effectively reintroducing the same problem as a standard behavioral design when scaled. Conversely, XOR-based and I-LVT methods, despite reducing some long combinational paths, required substantial BRAM duplication to accomplish the necessary number of parallel accesses, quickly exhausting available resources.

Instead, the proposed architecture merges the strengths of each methodology. Recognizing that the central challenge lies in determining where the most recent data resides, the solution is to always store the current value at a single location, whether in registers or other heterogeneous components, such as LUTRAM or BRAM. However, doing so reintroduces the issue of parallel access, which preemptively restricts the use of the latter. This effectively captures the essence of conventional register-based approaches consequently exhibiting the aforementioned limitations. Alternatively, to remedy this, the proposed CSR unit employs BRAMs for decoding and multiplexing operations, akin to the XOR-based approach, while storing the data in FFs. This is accomplished by implementing a one-hot decoding selection mechanism. Rather than storing data in the BRAMs, they are used as ROMs, solely for address decoding. When a specific CSR is accessed, only the corresponding output is activated, allowing the input or output of the FF to continue through the remainder of the read or write operation. Figure 3 visually depicts this mechanism.

Nevertheless, decoding the address alone is insufficient. The nature of the access—whether read, overwrite, bitwise set (OR), or clear (AND)—must also be resolved. To minimize fan-out and enhance routability, the proposed microarchitecture duplicates this mechanism, assigning each a distinct role in operation selection. Since read and write operations are independent, three replications of the BRAM decoding system are required: one to indicate the incidence of a read, and two to indicate the absence or presence of one of the remaining three functions. Without this, a single signal would need to drive extensive logic across the die, creating congestion and therefore significant timing penalties. Additionally, the system must support more CSRs (259 in the proposed architecture, as stated in Section 2.1) than can be handled by a single BRAM output (72 bits), necessitating a banked structure. This predicament is not trivial, as ensuring that only the desired CSR is activated per access requires careful address mapping within these banks, along with a robust reset mechanism to guarantee single-output activation.

In order to facilitate comprehension of the underpinning architectural decisions and their impact on performance enhancement, several fundamental considerations must be elucidated. Firstly, as is widely known, BRAM accesses are notorious for their significant routing delays, for which the common solution is to register its output. Additionally, for the banked memories in charge of write operations, an extra cycle was incorporated into the design to minimize routing delay. Thus, for optimal performance, the pipeline integrating this CSR subsystem must contain at least five stages. This structure is illustrated in Figure 6, which presents an overview of the finalized design.

Secondly, it is important to clarify the mechanisms by which atomicity and synchronization emerge. As discussed in the preceding subsection, implementing atomicity by means of operating read and write access in the same cycle offers several advantages but also one problem: extending the number of operations executed within a single cycle. One salient benefit of the methodology under examination is its capacity for pipelining operations across multiple cycles, a feature that distinguishes it from the behavioral multiplexing approach, which is largely constrained to operate on a single cycle. This enhanced flexibility results in a substantial reduction in the critical path, as will be demonstrated in Section 5.1. The fundamental reason for this phenomenon is that the only process that cannot be subdivided into multiple cycles is the actual access itself, as this could potentially result in reading an outdated value. However, the decoding and multiplexing processes can be separated without compromising integrity. The behavioral approach is inherently limited in its ability to separate these operations, and if it were to be attempted, it would require even more resources. In contrast, the proposed mechanism of decoding via the BRAM effectively enables their disentanglement from the actual access, with the corresponding enhancement in performance.

Additionally, one final optimization that can be attained with BRAMs involves their utilization to access every read-only CSR, as well as those only modified by CSR instructions, and thus, requiring no parallel access by the processor, such as *mhartid*, *mimpid*, and so on. In the figures, these are referred to as nominal CSR BRAMs. Their access would be equivalent to a normal register file, the data would be stored inside BRAM, and for reading, the output would be multiplexed with the other FFs, as will be elaborated below. This can constitute a non-trivial enhancement in specific processors if the quantity of these registers is large enough, thereby reducing FF usage and congestion. Nevertheless, if the number of CSRs accessed in this manner was negligible, this portion of the design could be omitted, as the increased complexity might not justify the marginal gain, and, in certain scenarios, it could even degrade performance due to the timing characteristics of BRAM.

Beyond BRAMs and FFs, the next components integrated into the proposed CSR subsystem are LUTs and CARRY modules. LUTs fulfill two distinct functions. Firstly, to determine the value to be written into FFs during the writing process. As previously stated, each group of BRAMs identifies whether a specific operation (read, write, set or clear) should be performed. The actual operation, however, is executed within the LUTs.

Secondly, LUTs are also used to select the value during read operations, with a smart optimization employing CARRY modules. As discussed, the write and read operations must occur in the same cycle to preserve atomicity. During a read operation, the final action to be completed is the muxing of the different values held by each FF, alongside with the values of the nominal CSRs and normal register writes to **General-Purpose Registers (GPRs)**. Considering the delay associated with this process, the optimization is as follows: as the data to be read from the CSR and written into the GPR is the former value of the CSR, in the worst case scenario, this value is settled at the end of the write of the previous cycle.

Thus, the read multiplexer selection can be partially computed during that cycle without compromising the integrity of the data, increasing timing slack in the next cycle. The utilization of the CARRY module in conjunction with dual-output LUTs constitutes an effective solution for

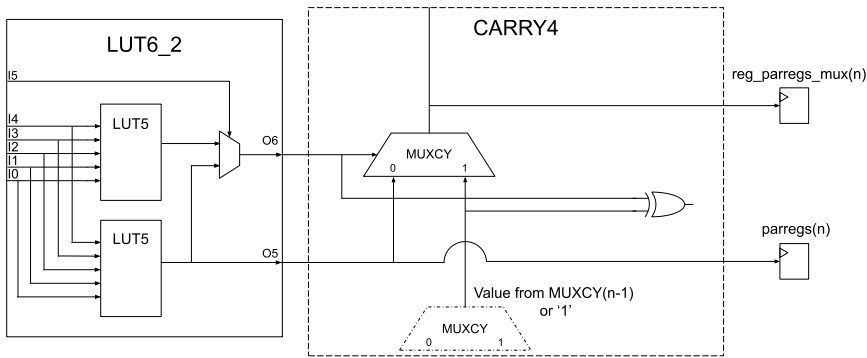


Fig. 4. Top portion of a slice. The inputs for each LUT are as follows. I0: old value from $\text{parreg}(n)$, I1: new value in the event of a parallel write, I2: new value, or mask, in case of a nominal write operation—or control for the parallel write in case I3 and I4 are “0,” I3 and I4: selectors from the BRAM decoders for the three kinds of write operation, I5: selector in the event of a read. The BRAM decoder for read operations functions with active-low logic. Consequently, when I5 is “0,” in the next cycle, this CSR will be read, and therefore, O6 mirrors O5, obtaining the desired output in the multiplexer FF. Conversely, when I5 is “1” the output from O6 is always “1,” masking O5 and allowing the value from $\text{MUXCY}(n-1)$ to propagate. The default value in case no output from a slice is selected is “1.” Additionally, since the BRAM decoder is always one-hot, this guarantees the propagation of the correct value. This unconventional optimization serves as a prime example of when instantiation is appropriate, as inferring such intricate structure would prove exceedingly difficult.

this purpose, as it enables an extremely optimized packing strategy and the execution of this multiplexing operation within the same slice, thereby minimizing delay. Figure 4 shows the layout of the top portion of such a slice. Furthermore, Figure 6, aims to clarify the distribution of these elements across the broader system, along with a depiction of the operations transpiring in each of the different stages.

The incorporation of the CARRY module has allowed the reduction of existing data to multiplex by a factor of four, from 259 to 65. Nevertheless, this would still necessitate multiple logical levels to be multiplexed via LUTs, encompassing routing paths traversing at least five slices with the associated delays, as discussed in Figure 15 of [25]. Furthermore, this phenomenon would need to be replicated across the entirety of the register width, ultimately resulting in a significant consumption of area. Nonetheless, this approach was implemented to allow for a comparative analysis with alternative proposals. As outlined at the end of Section 2.2, several techniques exist for producing efficient multiplexing. Three of these methods were implemented: muxing via LUTs, utilizing OR operations with a one-hot mask, and the approach that demonstrated superior outcomes in every metric, via DSPs. Information regarding performance, area, and power consumption can be found in Section 5, where they are designated as LUTmux, ORMux, and DSPmux, respectively.

Lastly, attention is directed toward the DSP, which constitutes the final primitive integrated into the design. A detailed exposition of the multiplexing implementation within this resource is subsequently presented. The primary advantage of utilizing the DSP lies in its ability to avoid signal routing across multiple slices, consolidating the multiplexing operation within a single component.

Leveraging the DSP's wide data-path capabilities—with three input vectors, A, B, and C, sized at 30, 18, and 48 bits, respectively—allows multiplexing up to 96 signals within a single clock cycle. The data are fed from the fabric registers, and then multiplexed combinationally, arriving at the GPR file where it is finally registered. This scheme is then replicated across the entire register width, for example, 32 times in a processor with XLEN = 32. Given the underutilization of these components in softcore processors, the impact is minimal. Subsequently, since the one-hot selector

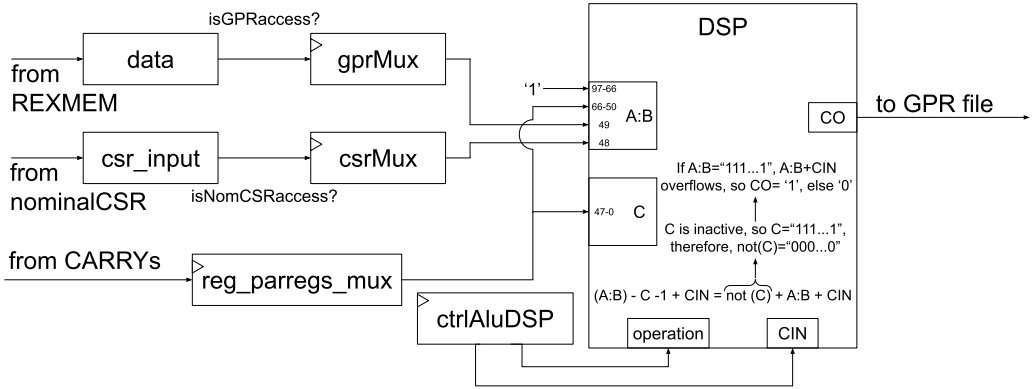


Fig. 5. Example of using DSPs for one-hot signal multiplexing in the CSR subsystem. It is important to note that the signals from the nominal CSR and normal register operations need to also be masked, as represented by the interrogation sign in the figure. From the arithmetic operations in [26], those permitted are: $\text{not}(C) + A:B + \text{CIN}$, as shown in the example (with $\text{CIN} = 1$, to allow potential trigger of the overflow), and $\text{not}(\text{not}(C) + A:B + \text{CIN})$.

signal generated by the BRAMs guarantees that only a single input is active at any moment, the correct output value can be computed through an arithmetic operation, with the carry output representing the selected signal. In this manner, the multiplexed result is efficiently extracted, without requiring additional logic except for producing `ctrlAluDSP`. The choice of operation at each moment is non-trivial and varies depending on the specific input data, but the objective is to always allow the one-hot active bit to decide whether overflow occurs or not. To aid comprehension of the underlying mechanism, Figure 5 exhibits a representative example that illustrates the general principle.

Now, a brief discussion regarding the applicability of this design in other processor architectures will ensue. As depicted in Figure 6, the pipeline responsible for hosting the CSR subsystem emulates a canonical five-stage RISC design (IF, ID, EX, MEM, WB). The rationale behind this approach is twofold. Firstly, its structure mirrors that of the processor from which this work originated. Secondly, five stages are sufficient to allow for adequate action segmentation, thus representing the minimum latency required to achieve notable performance enhancement. As discussed earlier, BRAM access routing delays typically necessitate registering BRAM outputs, introducing latency for signal propagation onto the DSPs; hence, a pipeline of at least five stages.

However, it should be noted that the surrounding environment does not actually implement the full processor. As will be elaborated further in Section 4, the test environment solely emulates the processor. Although the work is rooted in a processor with RV32I base ISA and the Zicsr extension, as pertinent to the context of this document, the execution units were removed, as well as any other control logic unrelated to CSR access. This enabled the isolation of the timing and routing behavior intrinsic to the CSR subsystem without interference from other datapath elements. The five-stage configuration is therefore not a simplification, but rather a controlled environment that ensures that all observed effects arise from CSR interactions themselves.

Consequently, the design principles formulated in this study are agnostic to pipeline depth, being the only prerequisites for adopting the proposed CSR subsystem to support decoding of CSR instructions (Zicsr extension) and sufficient latency budget to accommodate the microarchitectural changes outlined. A prime illustration of this is the configuration proposed that attains maximum performance (250 MHz), whereby an additional cycle is incorporated to enhance the

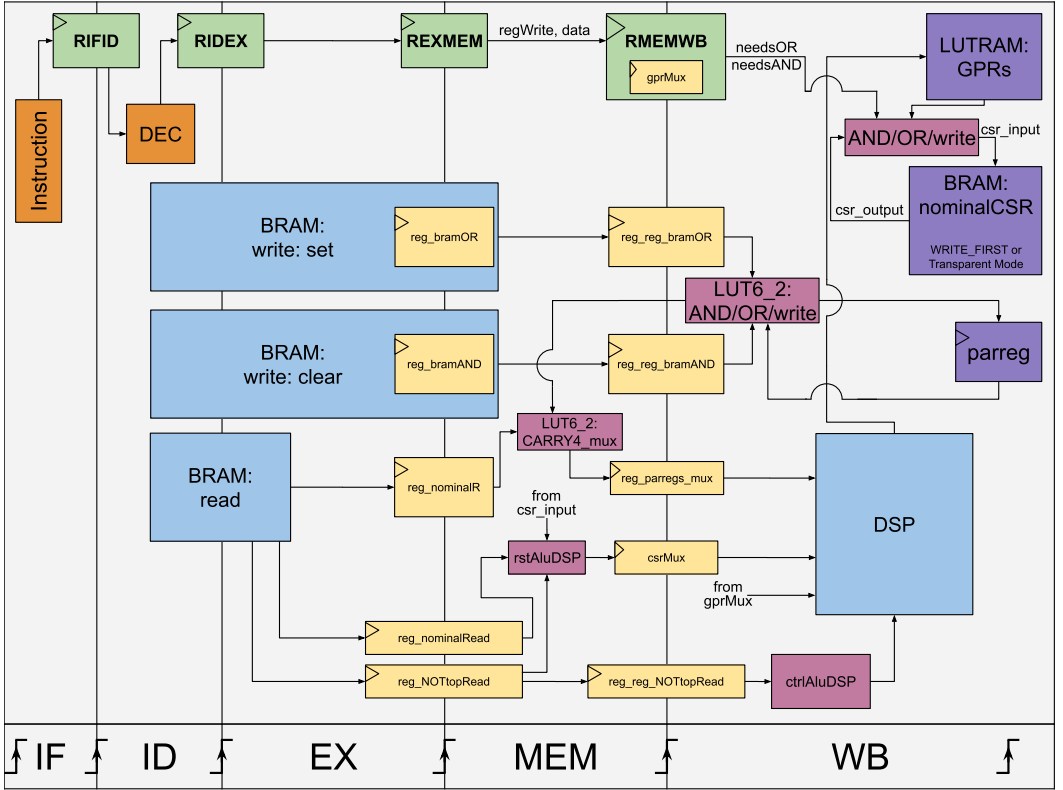


Fig. 6. Full depiction of the proposed CSR subsystem, excluding the component responsible for emulating the parallel interface with the processor. This component would have been placed adjacent to the “parreg” module. A visual representation of the actions occurring in each stage is provided. Notably, unlike other processors, which operate the register file at the opposite edge to reduce latency, in this microarchitecture, every component functions at the rising edge. This provides each path with the maximum possible slack, alleviating the burden on the router. Additionally, the selection of the operation to be performed by the DSP is illustrated. During a read operation, the corresponding BRAM decoder generates a signal indicating whether the read is located in the top portion of the DSP, thereby determining the required operation.

signal propagation. This is further discussed in Section 5.1. Hence, extending the microarchitecture to even deeper architectures, such as those present in Rocket or BOOM, would primarily involve locating the commit or retire boundary and aligning the CSR update accordingly. As discussed, introducing one additional stage could even be advantageous in certain contexts. Conducting such an integration, together with its associated timing characterization and other analyses, constitutes a natural continuation for future work.

To recapitulate, referring back to Figure 1 at the beginning of Section 3, the goal was to minimize the actions executed in a cycle, but always considering the underlying hardware generated. As such, although this design involves a greater number of actions within a cycle compared to the best behavioral implementation, it still exhibits superior timing characteristics, as will be evaluated in Section 5. The latter involved decoding the address, multiplexing the output, and both updates in the final cycle, whereas the proposed design avoids the delay in decoding, instead including the extraction and calculation of the new data, with muxing and updating being indivisible for atomic access. The remaining actions are conducted in different cycles. Of particular note is the detection

of access to an illegal CSR, which is accomplished through bit 71 of the BRAMs, as elucidated in Figure 3.

The rationale behind this design choice stems from needing a LUT for each FF to determine whether the update occurs “nominally” or in parallel. This allows using that LUT to seamlessly integrate the calculation of the new value, while handling the multiplexing of the output via the DSPs. Therefore, the proposed hardware-aware alternative exhibits the additional advantage of circumventing hazard detection, avoiding stalls of the pipeline flow and the resources associated with it; all while maintaining a critical path of a single LUT and a CARRY4, as seen in Figure 2(e).

3.3 Synthesis and Implementation Considerations

When seeking to optimize a digital design, the QoR achieved by any FPGA implementation is influenced by a wide range of factors. As is frequently emphasized in digital design methodology, optimizations applied earlier in the development process tend to have a more profound and far-reaching impact. For this reason, improving the RTL description should always be the first step, as has been the focus of the preceding subsections. Beyond architectural and behavioral considerations, further improvements can be achieved during the synthesis and implementation phases with the judicious use of a variety of strategies [27]. This subsection briefly outlines some of them, whilst their impact is analyzed in Section 5. In addition, it should be noted that although the focus of this work is on the integration and optimization of a CSR subsystem for a softcore processor, the considerations discussed herein are broadly applicable to high-performance or resource-constrained digital systems implemented on reconfigurable platforms.

The initial of these factors to consider is the configuration and fine-tuning of tool-level options, such as synthesis and implementation directives. These parameters influence, among other aspects, toolchain effort levels or place-and-route strategies, thereby impacting the timing closure, resource utilization, and power profile of the design. Regarding synthesis, most FPGA toolchains offer optimization levels (e.g., *performance high*, *area optimized*, *balanced*) that govern the intensity of logic restructuring, replication, and register retiming. Similarly, placement and routing directives instruct back-end tools on where optimization efforts should be concentrated during this stage of the flow. For instance, congestion-driven placement can mitigate routing-induced delay in dense logic clusters, while other options are more effective in designs with timing bottlenecks induced not by computation but by fanout, path divergence, or resource sharing. It is important to note that these configuration parameters do not alter the functional behavior of the design. Rather, they influence heuristics and algorithms employed by the toolchain concerning optimization priorities and acceptable tradeoffs. As evidenced by the implementation flow of the CSR subsystem in Section 5, augmenting optimization effort yields substantial reductions in critical path delay, reduced logic congestion, or enhanced register packing, albeit at the cost of increased compilation time and resource consumption, due to logic duplication aimed at favoring shorter routing paths.

Furthermore, the employment of floorplanning and manual placement techniques has the potential to further influence QoR of implementation results, particularly in complex, heterogeneous, or high-utilization designs. While modern FPGAs provide sophisticated placement engines, uncontrolled logic distribution can result in suboptimal placement, long routing delays, and clock skew. The first technique is manual placement, which although not recommended for large portions of a design, can provide significant benefits when applied to smaller segments, or with large modules like BRAMs or DSPs. Alternatively, higher effort directives can serve to achieve satisfactory placement and then freeze it for future implementations. Floorplanning, in contrast, is a related technique that instructs the placement tool that certain parts of a design belong together, while allowing more flexibility to be exercised during placement. By grouping functionally related modules and confining them to a specific region, one can significantly improve the engine’s search space, facilitating the

work of its heuristics and resulting in improved spatial coherence, reduced routing congestion, and easier timing closure. Beyond improvements in maximum clock frequency and resource and power utilization, these techniques can also enhance timing predictability and routing efficiency, especially when combined with incremental compilation flows. In such cases, placement consistency can be leveraged to shorten build times and preserve timing across iterative design cycles.

Finally, another impactful technique is constraint tuning. While the primary clock constraint defines the global timing objective, it is possible to specify more nuanced constraints, such as false paths, multicycle paths, or maximum delay exceptions, to name a few. These allow designers to tailor the synthesis and implementation processes to better reflect architectural intent. Among these, overconstraining stands out as a deliberate strategy to improve timing closure in challenging designs. This involves intentionally tightening certain timing requirements, typically by reducing the clock period or increasing clock uncertainty, in order to apply additional optimization pressure on the most critical paths. The practice of overconstraining a design is sometimes discouraged because of the risk of misleading the optimization engine, which could lead to inefficient resource allocation, unnecessarily prolonged tool runtimes, and potentially degraded performance. However, when applied judiciously, particularly in conjunction with path-specific analysis and floorplanning, this technique can improve timing closure by directing the tool's efforts toward specific logic regions that might otherwise receive insufficient attention due to an underestimated delay.

In summary, although synthesis and implementation considerations may initially appear unrelated to architectural design, they exert substantial influence on the practical viability of high-performance FPGA systems. Collectively, these factors influence not only timing and area utilization, but also power consumption, routing congestion, and, in general, the overall effort required to achieve design closure. Consequently, their incorporation into the design methodology is not solely advantageous but often imperative in propelling designs beyond their baseline operating range and into regimes suitable for demanding application domains, such as embedded and real-time environments. Lastly, it is important to emphasize that these techniques must be empirically driven. The employment of optimization directives and physical constraints must be substantiated by a rigorous methodology, grounded in adequate application of timing analysis, critical path evaluation, and examination of the relevant corresponding implementation reports, such as resource utilization reports, rather than being applied indiscriminately. The efficacy of these techniques is intrinsically linked to the structural properties of the design and the nature of the specific critical paths. In an era increasingly shaped by abstraction and automation, this underscores the enduring importance of architectural insight and designer expertise within an iterative, analysis-driven refinement process.

4 Verification and Validation of the Design

A rigorous and systematic verification and validation process is especially imperative to ensure the functional correctness and integration readiness of designs involving complex control and datapath interactions, such as the proposed CSR subsystem. This section outlines the methods applied to verify that the microarchitecture satisfied its defined performance and reliability criteria. The evaluation combined simulation-based verification and on-board testing, providing complementary evidence of reliable operation at the target frequencies.

The CSR subsystem was implemented in VHDL and verified independently from a complete processor core to enable rapid microarchitectural exploration. As detailed in Sections 3 and 5, many diverse designs were implemented, thus rendering crucial the ability to swiftly iterate. For this purpose, the testing environment was established employing a minimal ecosystem, intentionally decoupling the CSR microarchitecture from a full processor to isolate the timing paths exclusively related to CSR accesses, as was the goal in this work. This decision also minimized confounding variables and accelerated fault identification, which would have been considerably more difficult in

a fully integrated system. Thus, the dedicated test environment was developed to model processor interactions with the CSRs. It incorporated a minimal instruction decoder, general-purpose registers, a lightweight pipeline, and a parallel interface that emulated communication of the CSRs with the processor to achieve their specific functionality, as shown in Figure 6.

Functional correctness was assessed using the experimental setup described in Section 5. Testing was divided in two phases. The first involved sequential CSR accesses, where instructions targeting different CSRs were issued in programmatic order, testing the correct data routing and write-back functionality under normal pipeline flow. The second phase tested parallel CSR accesses, with the testbench performing simultaneous reads or writes to multiple CSRs. These emulated concurrent interactions typical in processor architectures, such as exception handling and performance monitoring logic engaging the CSR bank simultaneously.

The test suite evaluated the CSR instructions defined by the ratified Zicsr extension, including CSRRW, CSRRS, and CSRRC. The immediate variants of these instructions (CSRRWI, CSRRSI, CSRRCI) were not tested, as they affect only operand decoding (e.g., sign-extension of immediates) and do not impact the CSR access logic. The instructions were tested under the following conditions: To maximize test determinism and simplify traceability, deterministic values were used. Each CSR was written with a value equal to its address minus one. This scheme allowed straightforward visual verification in waveform inspection. In later test iterations, this was complemented with assertion-based checks, verifying that read-back values matched expected results at each cycle. All 4,096 possible CSR addresses were accessed during tests. Both assigned and invalid addresses were included to test the full decoding path and ensure robustness of the access multiplexer logic.

Post-synthesis and post-implementation simulations verified that functional correctness was preserved after each stage on the flow, and static timing analysis revealed no critical path violations under the target operating frequencies. A more detailed evaluation on performance, area, and power is provided in the following section. Lastly, to verify hardware operation on the FPGA without incurring in the timing overhead of Integrated Logic Analyzer instrumentation, a hardware self-checking mechanism was implemented. A finite-state machine iterated over all CSR addresses with an internal counter, verifying expected results in real time. Successful operation was indicated by a heartbeat LED toggle, while error conditions froze this indicator and activated an error LED.

While this test setup does not encompass full system integration, it provides a robust validation of the CSR subsystem under realistic timing constraints. Future work will include integration with our full RISC-V core to capture system-level interactions.

5 Results

This section presents a comparative evaluation of the proposed RISC-V CSR design with respect to a conventional behavioral approach. The analysis focuses on three key metrics: timing performance, resource utilization, and power consumption. While the implications of these results are alluded to in each subsection, a more detailed analysis is provided in Section 5.4. Prior to this, a brief discussion of the experimental framework and the applicability of the findings will now be conducted.

As mentioned during the introduction, all experiments were performed on a Digilent Nexys A7 development board [11], featuring a AMD Xilinx Artix-7 FPGA (xc7a100t-1csg324). Notably, this device corresponds to the lowest speed grade available for the Artix-7 family (speed grade-1), imposing more stringent timing constraints and thus rendering the performance results obtained more conservative and representative of worst-case hardware conditions. The design flow was implemented using AMD's Vivado Design Suite, version 2022.2. It should be emphasized that the observed performance metrics—maximum operating frequency (Fmax), resource utilization, and power consumption—are inherently dependent on the underlying FPGA architecture. They can be influenced by factors such as the logic fabric granularity, the utilization of FPGA-specific

primitives like DSPs or BRAM, its clocking structure, and specifics of synthesis and place-and-route strategies, which in turn depend on the constraints applied and family-specific layout rules. These considerations underscore the complexity of FPGA performance optimization, which involves balancing tradeoffs among speed, area, and power under architecture-specific restrictions.

For example, in the 7-series FPGA family, all FFs within a slice share a single control signal. This limitation reduces the amount of logic that can be allocated together, thereby diminishing design compactness and increasing net delays, which negatively impacts performance. Nonetheless, as vendors strive to evolve their products, newer architectures exhibit greater versatility and flexibility in logic and resource allocation, while largely maintaining backwards compatibility. Consequently, despite the fact that the results presented here are based on the Artix-7 FPGA, the techniques and insights gained from this evaluation are likely to be applicable to other architectures and vendors, albeit maybe with certain modifications.

For instance, more modern AMD Xilinx architectures, such as UltraScale and UltraScale+, no longer employ the DSP48E1 primitive used in the 7-series family [26]. Instead, they incorporate an evolution on the former, the DSP48E2 slice [28], featuring extended input widths, improved pre-adder capabilities, and additional functionality; while maintaining backwards compatibility with respect to the functionality described in this article. Notably, this compatibility extends even to their latest architecture, Versal, employing the DSP58 [29], which further augments the capabilities of its predecessors while preserving their fundamental operational semantics. Likewise, a similar argument can be made about the block RAM primitives. While newer primitives, such as RAMB18E2 in UltraScale [30], or RAMB18E5 [31] in Versal, exhibit minor functional and configurational differences, the fundamental memory organization and operational principles remain compatible with the requirements of the CSR subsystem presented herein.

More generally, regarding applicability with architectures from other vendors, the most crucial aspect would be the width of the various primitives employed. To illustrate this point, consider the BRAM decoder. This architectural concept enhances the design, facilitating one-hot decoding of numerous addresses in a single unit. Consequently, if width is reduced, the degree of enhancement may be diminished, or the design may necessitate modification. An example of this is found in Intel's FPGAs, where one of the most common RAM memory blocks, M20K [32], only allows port widths of up to 40 bits [33], and thus demands additional units to achieve equivalent functionality. This could be mitigated reducing the duplication of resources (as currently is done at each access type, i.e., write, set, or clear), and through the use of other less common primitives, such as M144K [34].

The same rationale could be posited for the concept of multiplexing via DSPs, although in this case, most contemporary devices of the major brands are equipped with reasonably wide units. However, a potential limitation exists concerning the flexibility of the DSP primitives, which may not be as adaptable as those in Xilinx architectures. For instance, although Intel's DSPs offer widths of over 100 bits [35, 36], they are primarily designed for multiply-accumulate operations. Therefore implementing multiplexing may require use of architecture-specific instantiation modes to effectively bypass multiplication. In practice, some operands may need to be constrained to the decimal value of "1," neutralizing multiplication, and allowing the actual desired operation to be completed within the adder unit. Hence, despite the remaining width probably being sufficient (72 bits in pre-adder mode), the presence of the multiplication datapath may still incur some delay in the critical path.

5.1 Performance Results

Performance was evaluated based on the minimum clock period and maximum frequency obtained for each design. For this purpose, an optimistic target frequency was initially established in Vivado's Clock Wizard, and timing closure was assessed. When closure was not achieved, the target frequency

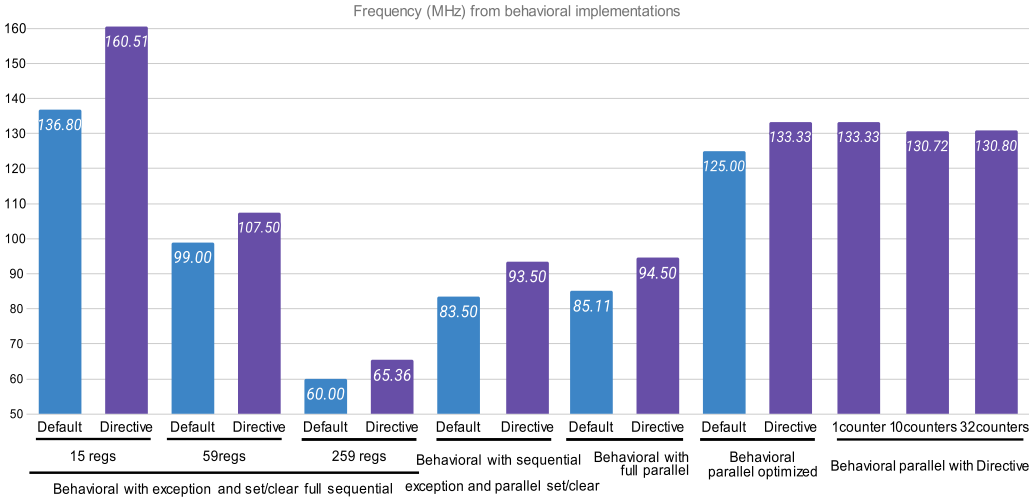


Fig. 7. Performance results for each of the examined behavioral implementations. Distinct colors indicate the specific optimizations applied in each configuration. Note the starting point of the vertical axis selected to accentuate the relative variation between configurations. For a comparison in absolute terms, refer to Figure 12.

was iteratively reduced until a successful implementation met the timing constraints. Figures 7 and 8 summarize the performance results, providing a comprehensive graphical overview across all tested configurations. Due to the density of information displayed, additional comparative figures focusing on selected key trends and comparisons are provided in the supplementary material, along with a complete set of numerical results in tabular form. This organization aims to enhance readability in the main text while preserving detailed insight into specific comparisons for interested readers.

In summary, the best implementation from the proposed design achieved a significant reduction in clock period, reaching 4 ns compared to 16.667 ns in the worst behavioral implementation, corresponding to an increase in Fmax from 60 to 250 MHz (a 316,667% improvement). In the following paragraphs, an in-depth analysis of these results is elaborated. To ensure fair comparison, multiple implementations were conducted, reflecting the different optimization strategies discussed previously. This also allowed observation of how Vivado responds to varying design characteristics. Reported results correspond to the highest-performing configuration for each case, that is, the point beyond which further frequency increases prevented successful timing closure, although perhaps manual intervention might have yielded minor improvements.

The first notable finding is how performance degrades with additional CSRs in behavioral designs. This is shown in the six leftmost columns of Figure 7, and Figure S1 of the supplementary material, where Fmax of a 259-register implementation drops to 43.86% of that achieved with only 15 CSRs under default implementation strategies; from 136.80 to 60 MHz. This reduction is even more apparent, down to 40.72%, if a higher effort directive was used; from 160.51 to 65.36 MHz. This trend explains why several processors from Section 2 of the supplementary material experienced comparatively reduced performance loss, as they implemented only a limited number of CSRs. A related observation is that as the number of registers decreases, the improvement in performance with implementation directives different from the default increases. While not guaranteed to always hold true, in general, since these directives offer a tradeoff, sacrificing longer execution time in

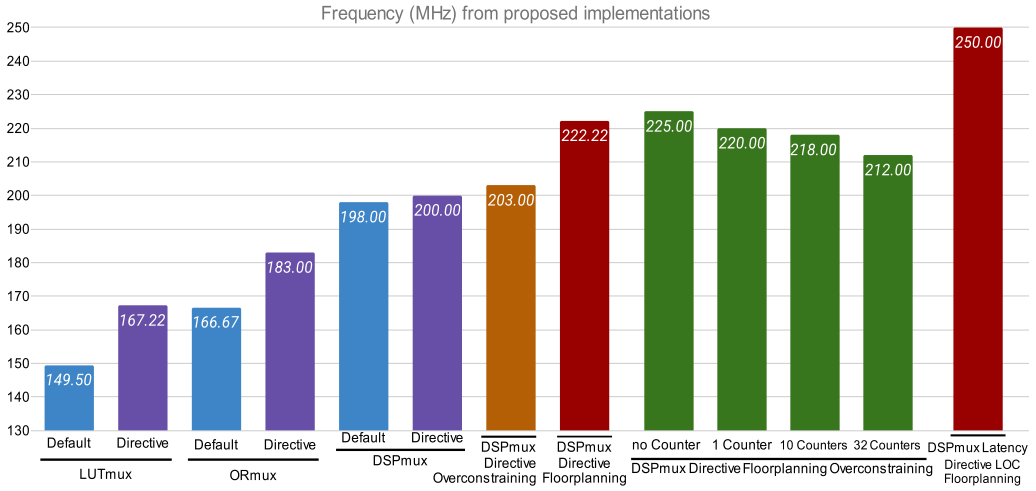


Fig. 8. Performance results of the proposed implementations. Distinct colors enable comparison between optimizations: blue (default implementation directive), violet (higher effort directive), orange (effort directive and overconstraining), red (effort directive and floorplanning), and green (all three optimizations).

exchange for higher performance; with fewer resources utilized, it is possible to search a wider area of the solution space, arriving at a superior netlist placement, rather than settling at a local minima.

Secondly, the subsequent columns of Figure 7 illustrate the results regarding the optimizations of behavioral implementations discussed in Section 3.1. Particularized results for these configurations are showcased in Figure S2 of the supplemental material. For example, relocating computation of the input value for set/clear instructions to earlier pipeline stages increases performance 39.17%. That is, 139.17% relative to the original design under default synthesis strategies (83.50 MHz), and 143.05% using more aggressive implementation directives (93.50 MHz). In contrast, modifying the stage of evaluation for exception conditions yields smaller improvements: approximately 1–2%. This brings overall performance to 141.84% and 144.58% compared to the fully sequential behavioral baseline, respectively, 85.11 and 94.50 MHz in comparison to the starting point of 60 and 65.36 MHz.

Despite the previously applied optimizations, the critical path in the behavioral implementations remained greater than the theoretical minimum of five logic levels, established in Section 2.1. After addressing synthesis inefficiencies, as detailed in Section 3.1, a refined implementation was produced with eight logic levels, approaching the theoretical limit given the sparsity of the CSR address space. This adjustment resulted in a 46.88% improvement over the prior fully parallel behavioral design when using default implementation strategies, and 41.09% with higher-effort directives; specifically reaching 125 and 133.33 MHz. Relative to the original naive behavioral baseline, this corresponds to a total improvement of 108.33% and 104.00%, respectively.

Following the evaluation of the behavioral implementations, this analysis proceeds to compare the performance between the different proposed microarchitectures. These are exhibited in Figure 8. As described at the end Section 3.2, the main implementation employs a multiplexer strategy based on DSP components. However, for completeness, a design of each of the other solutions is available for comparison, namely, behavioral multiplexing using CLB LUTs and F7/F8 muxes (LUTmux), an optimized OR-gate-based scheme (ORMux), and the eventual DSP-based approach (DSPmux). Nonetheless, it should be noted that the first two implementations do not pass all functional tests, as they were developed during earlier design stages for exploratory purposes, and were later

made obsolete with the development of the latter design. Still, their inclusion remains valuable for assessing the relative performance potential of different multiplexing strategies. Although small variations are possible, the modifications required to make them fully compatible are minimal, allowing for a meaningful comparison with the final implementation.

Performance of these implementations can be studied in the six leftmost columns of Figure 8, and Figure S3 of the supplemental material. Among the designs, LUTmux exhibits the lowest performance, followed by ORmux, while DSPmux achieves the highest Fmax across all configurations. Nevertheless, all three designs demonstrate superior performance compared to fully behavioral approaches. Under default implementation directives, ORmux outperforms LUTmux by 11.48%, whereas the difference reduces to 9.44% with high-effort directives, corresponding to a Fmax variation of 17.17 and 15.78 MHz, respectively. In addition, the DSP design presents a 32.44% improvement with regard to the LUTmux implementation and 18.80% with ORmux under the default directive, while a 19.60% and 9.29% using the best directive. The comparatively modest improvement of the DSPmux design under high-effort optimization may be attributed to the difficulty in placing large components, such as DSPs and BRAMs. In designs incorporating such elements, manual placement or floorplanning is often recommended, to alleviate the difficulty of the task [27].

Once established that DSPmux was the best performing design, an attempt was made to determine the extent to which it could be improved. These correspond to the middle columns in Figure 8, and Figure S4 of the supplemental materials, in which the frequency of the proposed design with different implementation optimization strategies can be observed. As shown, the improvement using directives different from the default one is only 1.01% (from 198 to 200 MHz), and overconstraining the design merely resulted in a 1.50% enhancement (203 MHz). However, as previously noted, incorporating floorplanning, in conjunction with manual placement of DSP blocks, yielded a substantial improvement of 11.11% relative to solely employing a higher effort directive (222.22 MHz). Moreover, combined floorplanning and overconstraining improved Fmax until 225 MHz, 13.64% better than using the default directive and a 12.50% improvement over the best directive.

Furthermore, one additional optimization exists, given that the critical path in this latter design involves traversing the DSP and arriving at the GPRs within a single cycle. If the processor architecture is such that it could withstand one more cycle of latency with two write-back stages, writing the CSRs in the fifth stage and the GPRs in a hypothetical sixth stage, it would be possible to use the FFs from within the DSP to reduce congestion and routing delay considerably. Therefore, in this case even just using a higher effort directive and manual placement of the DSP blocks was enough to attain 250 MHz, which is 25% over the equivalent design optimization wise without the additional latency, and 11.11% over the best design featuring all available optimizations.

Having examined each design category independently, a comparison is presented between the best-performing behavioral and proposed implementations. This comparison is possible by examining Figures 7 and 8, although caution is required when interpreting them due to differing vertical axis scales. For a direct and unified representation, Figure S5 in the supplementary material consolidates the results. To ensure a fair comparison, only configurations employing default and high-effort implementation directives are considered, as these are consistently available across both design categories. As the results indicate, all DSPmux variants consistently outperform their behavioral counterparts by at least 50%. Specifically, the maximum operating frequency of the DSPmux design is 158.00% of that of the behavioral design under default directives, and 150.00% under the high-effort directive. In absolute terms, the highest frequency achieved by the behavioral implementation is 133.33 MHz, compared to exactly 200 MHz for the DSPmux design.

Lastly, the concluding performance comparison pertains to the presence of monitoring counters. Based on our prior experience with these counters, they were anticipated to be among the most

challenging CSRs to incorporate in this microarchitecture. Those effects are illustrated in the rightmost columns of the preceding plots and in Figure S6, in the supplementary material, which depict timing results for varying numbers of implemented counters. As observed, the proposed implementation experiences a small penalty when the number of counters approaches the maximum allowed by the RISC-V specification, whilst the behavioral design remains relatively unaffected.

There are two reasons attributed to this. Firstly, the proposed design is more compact, as explored in the next subsection, which can create congestion and hinder the router's ability to distribute signals across the die while meeting the tighter timing requirements. Secondly, the floorplan for the DSPmux design is organized so that each bit of the register width is multiplexed independently in a different DSP, and stored in a different LUTRAM, as described in Section 3.2. In contrast, behavioral designs concentrate related logic, as the calculation of the counter's new value occurs in cascading CARRY4 modules, in a single place of the die. Nonetheless, impact remains minor, with the maximum performance loss being 5.78% in the proposed design of 32 counters, compared to 1.96% in the behavioral implementation with 10 counters. Should this overhead become a critical concern, mitigation strategies are feasible. For example, distributing CARRY4 modules across the device, calculating the new value in multiple stages, provided that additional latency is acceptable, which, in the case of counters, is unlikely to pose a significant impediment.

Additionally, selection of high-effort placement directives is not static through the design process, as reflected in Tables S2 and S3. For example, behavioral implementations incorporating counters found two effective placement directives. Although certain heuristics can generally be applied, such as tightening delay estimates (e.g., ExtraNetDelay) when synthesis meets timing requirements but implementation falters, or spreading allocation when facing congestion (e.g., AltSpreadLogic), iterative experimentation remains essential to achieve optimal results. Finally, it is also worth noting that the test environment was intentionally configured as a worst-case scenario, wherein every single CSR is read and written in full width. In practical applications, such exhaustive access patterns are unlikely to occur, enhancing QoR by mitigating congestion and reducing net delays.

5.2 Resource Utilization Results

The impact on FPGA resource usage, including different LE, FFs, memory blocks, and DSP units, was evaluated among all proposed and baseline behavioral designs. A comparative overview of the three most relevant resource categories is illustrated in Figure 9, while the complete quantitative breakdown is available online as supplementary material in Tables S4 and S5. The ensuing analysis examines the observed variations across each major resource type.

Firstly, utilization of LE will be assessed, including LUTs, as well as F7 and F8 muxes. Among behavioral implementations, evidently, those with fewer CSRs exhibit reduced consumption of logic resources, mirroring the same trend for all other categories. Therefore, these configurations are omitted from Figure 9 to avoid misleading comparisons, but are reported in the supplementary material. Moreover, all implementations with default strategies consume fewer LUTs in comparison to the counterparts with higher effort directives. This disparity is likely due to optimization heuristics that replicate resources or disable LUT combining when anticipated to enhance timing closure.

With regard to the proposed designs, those employing multiplexing without DSP blocks show higher logic utilization, as muxing is performed within the CLBs using LUTs and F7/F8 multiplexers. In contrast, DSPmux implementations exhibit significantly lower LUT count, approaching the theoretical lower bound for storing 259 32-bit registers (8,288 LUTs), with some additional overhead attributable to the test environment. Among all evaluated configurations, these designs demonstrate the lowest overall LUT utilization. Specifically, DSPmux variants show reductions ranging from 18% to 55% compared to the behavioral implementations, as illustrated in Figure 9.

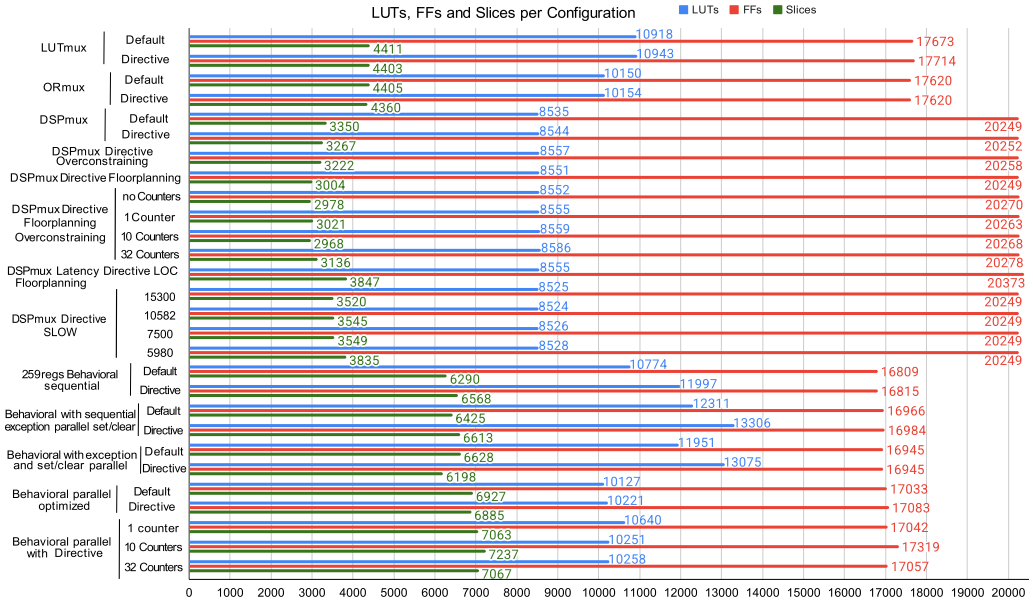


Fig. 9. FPGA resource utilization across all tested implementations. The bar chart illustrates the number of LUTs, FFs, and Slices used in each design. Although the proposed designs exhibit minor increased FF usage (approximately 15%), they achieve significantly more efficient packing, resulting in substantially reduced slice occupation, often halving it. The enhanced allocation efficiency is enabled by the heterogeneous design and optimized integration, exemplified earlier by the application of CARRY4 modules.

Secondly, albeit connected to this topic, DSP utilization will be evaluated. Given the reliance of DSPmux designs on these modules for data multiplexing, it is evident that they utilize DSP resources instead of the F7 and F8 muxes employed by the proposed designs with alternative multiplexing and all behavioral designs. Thirdly, regarding block RAM utilization, all implementations featuring the proposed decoding structure employ 14 BRAMs to facilitate the decoding of the CSRs, in contrast to their absence in behavioral implementations, as observed in the tables of the supplementary material. Specifically, as explained in Section 3.2, these BRAMs are organized as three banks of four, each dedicated to a distinct CSR access type, and two additional BRAMs to store nominal CSRs.

Fourthly, regarding FF occupancy, their allocation is higher in proposed implementations. This is a result of distributing CSR address decoding across multiple stages, as opposed to performing all operations sequentially in a single cycle, which demands storing intermediate data in additional FFs, as explained in Section 3.2. However, variation remains modest, with an increase of 15.92% between behavioral and DSPmux implementations employing default strategies. Moreover, it merits attention that, in a manner analogous with LUTs, yet in this case in the opposite direction, FF use approaches the minimum possible in behavioral designs, considering that the test environment duplicates the number of registers to emulate the parallel interface to the processor.

Finally, in an attempt to summarize general resource utilization across implementations, the number of occupied slices is often considered. In standard workflows, however, this measure is not always a reliable indicator of area efficiency, as Vivado is typically timing-driven, prioritizing performance over compactness. This can lead to logic being split or spread out to reduce congestion, with area optimization only applied explicitly or under conditions of high resource pressure [37]. Nevertheless, this metric still offers insight into how compact a design is, as it demonstrates how

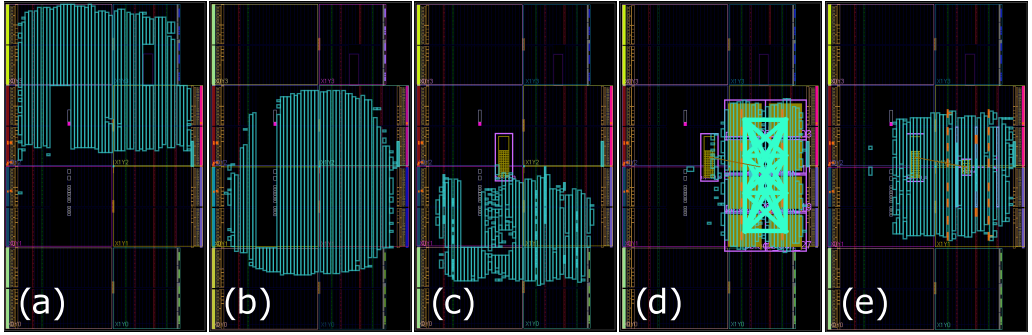


Fig. 10. Visual representation of FPGA area occupation for selected implementations. Designs are: (a) baseline behavioral; (b) optimized behavioral; (c) DSPmux—all three of them with default implementation directives; (d) DSPmux with high-effort directive, floorplanning, and overconstraining; and (e) DSPmux with added latency and minor manual placement and floorplanning. The behavioral implementations (a–b) occupy approximately twice the area of the DSPmux designs (c–e), spanning nearly four clock regions compared to just around two, illustrating the impact of design specialization and physical guidance on spatial efficiency.

readily Vivado can pack a given design into the die. As observed in Figure 9, the proposed designs exhibit lower slice utilization compared to the behavioral implementations. This is especially notable in the DSPmux designs where a reduction of more than half can be seen. This reduction is not inherently beneficial, as increased packing density can lead to congestion, timing closure issues, and degraded performance. However, as discussed in Section 5.1, the proposed designs yielded enhanced performance, indicating that the achieved compactness is indeed beneficial, as it translates into greater availability of FPGA resources for further development. Figure 10 illustrates this difference in utilization, highlighting the comparative spatial efficiency of the proposed implementations.

5.3 Power Utilization Results

In the following section, the energy consumption across all implementations is examined. First, it will be shown how the operating frequency exerts a fundamental influence on the energy consumption, followed by analysis in both relative and absolute terms of the corresponding power utilization of each design. To perform this analysis, a power supply was configured to deliver a voltage of 5V, as outlined in the Nexys A7 board manual [11]. Subsequently, the current drawn was measured, obtaining the power consumed accordingly. Measurements were conducted under two conditions: with the designs in the reset state, and when operational. The results are summarized in Figures 11 and 12, while the complete dataset is available in Table S6 of the online supplementary material.

A first attempt to compare power consumption across different implementations might involve measuring their total power directly without additional considerations. However, this would not result in a fair comparison, as one of the dominant factors in a digital system’s energy consumption is switching power, which arises from transistor state transitions between “0” and “1.” Switching power is highly dependent on the operating frequency of the design, as well as on the switching activity factor, which represents how regularly signals transition between logic levels [38]. Therefore, to ensure a meaningful comparison, all implementations should be evaluated while running at the same frequency, and subjected to the same battery of tests. Since the test environment is already equivalent for all designs, four additional implementations were created for the DSPmux design, each operating at a different frequency, to facilitate comparison with the other designs.

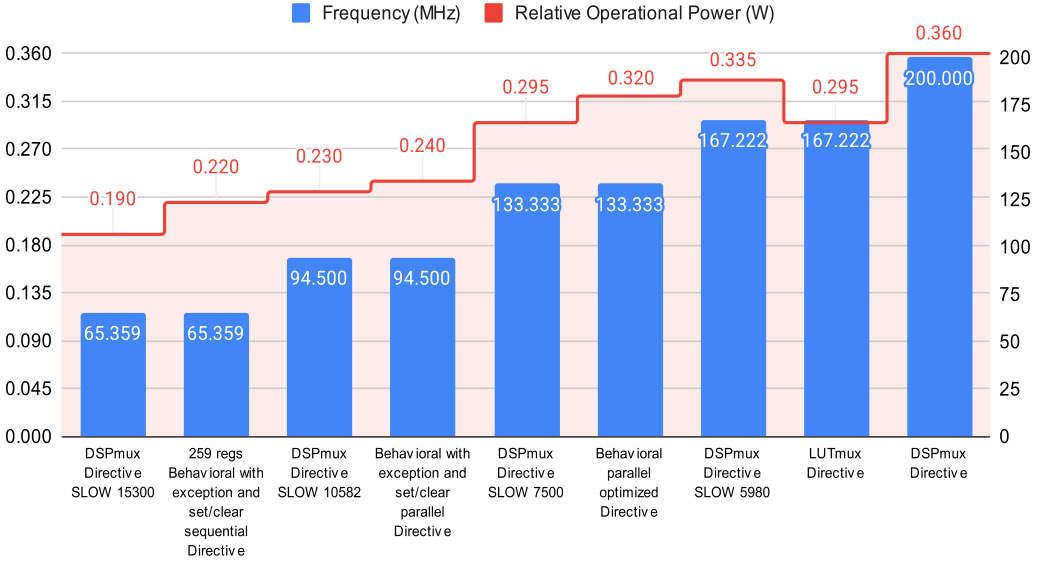


Fig. 11. Relative power consumption of several implementations at varying frequencies. Values are computed as the difference between the power drawn during active operation and that measured in the reset state. Pairs of implementations are shown, comparing power consumption at a fixed frequency, thereby demonstrating that behavioral implementations exhibit higher power consumption compared to proposed designs. Moreover, clear correlation between operating frequency and power usage is observed across designs.

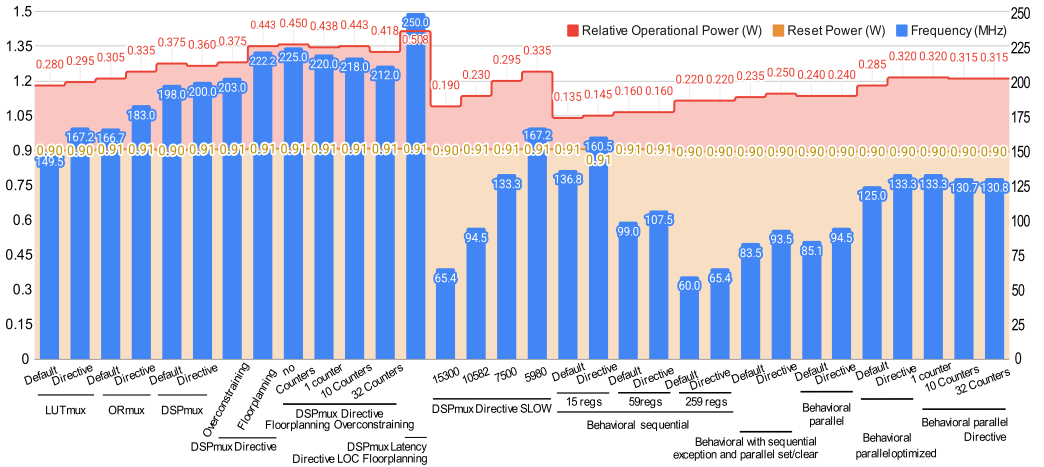


Fig. 12. Power consumption vs. Fmax for each implementation measured under full operating conditions. This includes switching activity from different modules, clocks, I/O, as well as static consumption such as leakage and fixed board components utilization, which remain active regardless of design activity and are excluded from design-level optimizations. Compared to behavioral implementations, the proposed designs exhibit greater power consumption (left vertical axis), attributable to their enhanced operational frequency (right axis).

In Figure 11, these tests are illustrated. As shown, DSPmux designs were configured to operate at the frequency of the worst and two best behavioral implementations, along with that of the LUTmux design. All four implementations were generated via higher effort directives and are labeled “slow” in all supplied tests. The objective behind this configuration is to provide a uniform distribution across the existing frequencies. Although the differences are minor, results favor the DSPmux design over behavioral implementations, and the LUTmux design over the former. Both outcomes align with the resource utilization data discussed in Section 5.2; the first due to the augmented switching from greater LUT usage, and the second because of increased FF utilization.

In this regard, as emphasized throughout this article, this work has focused on enhancing performance rather than minimizing power consumption. However, to achieve this, an extremely compact design was devised that efficiently manages FPGA resources, as discussed in previous sections. Despite the higher power density of DSPs and BRAMs compared to general fabric logic, they are significantly more energy-efficient for the operations they perform [26]. Consequently, by offloading functionality to these specialized units and reducing overall area and routing complexity, the total dynamic power consumption is lowered relative to equivalent behavioral implementations.

One final point to consider is that, in Figure 11, only the relative operational power consumption has been displayed, that is, the difference between the total power drawn during operation and the consumption while the system was in a reset state, serving as approximation of dynamic power. This method was chosen because it allowed a fairer comparison of the power used only during operation, since all other measurements remained virtually unchanged; however, when an understanding of the overall design is desired, it is necessary to examine the total power consumption. In contrast, Figure 12 depicts the total power consumption for every implementation analyzed. In this same figure, it can also be appreciated how the operating frequency exhibits a direct correlation with the power consumed. The reason for presenting both graphs is that looking exclusively at Figure 11 may give the misleading impression that the power consumption of the DSPmux implementation at 65 MHz is approximately half of that operating at 200 MHz, but this is only accurate if the static power dissipation and all other consumption produced by the board in the reset state are disregarded.

Looking at Figure 12 and Table S6, the maximum difference in power consumption is of 0.300 W, which represents a difference of 26.91%, but with a performance increase of 190 MHz, or 316.67%. Alternatively, a more equitable and reasonable evaluation could be made by comparing the best behavioral and DSPmux implementations with the same optimizations and latency requirements. In this scenario, there is a change in power consumption of 0.143 W, an increase of 11.73%, while in return yielding a performance improvement of 91.67 MHz or 68.75%.

Hence, it could be argued that, for the majority of practical applications, the increase in power consumption is negligible, and justified by the substantial improvements in performance. Nonetheless, for applications where the minimal power consumption is required, the proposed design would not need to be immediately discarded. Instead, there are many avenues which were not explored, since the focus, again, was on performance. For instance, access to FFs, block RAMs and DSPs could be gated when they were inactive. This would not necessarily imply a performance reduction resulting from an increased routing congestion if BUFG components were to be inserted [37].

5.4 Results Discussion

This comparative analysis has highlighted several key considerations between proposed CSR designs and conventional behavioral approaches. The most significant aspect is performance, where the best DSPmux implementation achieves a 316.67% improvement in maximum frequency compared to the behavioral baseline. However, this comparison reflects an extreme contrast, showcasing the best and worst configurations. A more realistic analysis involves evaluating both designs under similar

optimization conditions. This is because, as detailed in Section 5.1, applying different directives and techniques such as floorplanning, overconstraining or manual placement, can substantially affect implementation results. Nevertheless, when comparing the best behavioral approach to the proposed design under equivalent optimization strategies, the difference in frequency remains over 50%. Still, although the performance enhancement of the optimized designs is evident, with proposed implementations consistently outperforming behavioral models in terms of maximum operating frequency, the decision to adopt one approach over the other depends on multiple factors that are not limited exclusively to performance.

The most salient tradeoff concerns the compromise between QoR and design complexity and maintainability. The presented solution introduces additional architectural complexity, requiring deeper understanding of low-level optimizations and FPGA resource management. This complexity may pose challenges for teams prioritizing rapid prototyping, ease of debugging, or long-term maintainability. In contrast, behavioral implementations offer greater readability and flexibility, making it more suitable for iterative development and subsequent modifications. However, it should be noted that this does not preclude adaptability within the proposed design; it would simply require some time and effort to initially acclimate.

Another significant factor is scalability with CSR count. As demonstrated, behavioral approaches suffer noticeable performance degradation as the number of CSRs increases. This makes the proposed implementations particularly advantageous in systems with a high number of CSRs; in order to maintain reasonable performance within a scalable CSR microarchitecture. Conversely, in architectures with a limited number of CSRs, the performance penalty is less pronounced, and the simplicity of a behavioral implementation may outweigh the benefits of optimization.

A further aspect to consider is the critical path of the RISC-V processor as a whole. Depending on the overall system architecture, the critical path may not reside within the CSR module itself but rather in an unrelated part of the design. In such situations, the performance benefits derived from the optimizations discussed herein may have a reduced impact on system-level performance. This could diminish the relative advantage of the proposed approach, thereby rendering behavioral alternatives potentially more practical in non-performance-critical contexts, by simplifying the overall workflow and reducing design complexity.

However, when considering the impact of second-order effects on the overall performance of the processor, enhancements in the CSR subsystem can still yield indirect benefits at the system level. For instance, the roughly 50% decrease in slice utilization and the observed reduction in logic depth can alleviate congestion in adjacent regions of the processor, allowing implementation tools to focus more effectively on the actual critical paths. This can simplify timing closure and potentially improve the overall achievable Fmax, even when the CSR logic is not itself the limiting factor. This is an important nuance to consider when evaluating the broader implications of the tradeoffs between the two approaches.

Resource utilization and power consumption also influence the selection process. As discussed in the preceding sections, the proposed design exhibits reduced or comparable usage of FPGA resources relative to behavioral implementations, while featuring a significantly more compact layout that allows its packaging in a considerably smaller footprint. In terms of power consumption, the only notable increase is associated with the higher operating frequencies the presented design can sustain. When both approaches are evaluated under equal conditions, DSPmux actually demonstrates superior energy efficiency. This suggests that the architecture offers not only augmented performance but also the flexibility to operate more efficiently when lower speeds are sufficient. In power-constrained applications, designers could tradeoff excess frequency margin to reduce energy consumption, achieving a better performance-per-watt balance than what behavioral designs can offer. This flexibility allows the proposed design to adapt more effectively to diverse operational

requirements, whether prioritizing performance, area, or energy efficiency. Moreover, standard mitigation strategies such as clock gating or dynamic frequency scaling could be employed to further reduce energy usage without compromising performance.

Ultimately, the choice between methodologies depends on the specific requirements of the target application. Processor architects must navigate a tradeoff between QoR and simplicity. When optimization for performance is paramount, or in designs of greater scale with more restrictive area and power budgets, its ability to efficiently handle a large number of CSRs while maintaining high clock speed establishes the proposed solution as a compelling alternative. Conversely, when concerns for design simplicity, rapid implementation, as well as readability and maintainability take precedence, a behavioral implementation remains a viable and practical solution. This is because, as described during Section 3, many of the elements that get automatically generated by a behavioral implementation need to be precalculated on the proposed design. Although a behavioral design may not achieve the same level of performance, its straightforward nature and lower development overhead may make it more appropriate for applications with less stringent constraints.

As a potential future improvement, many of the downsides associated with the complexity of the proposed design could be abstracted away through automation. For example, a parameterized IPcore generator, implemented via a high-level scripting tool, could produce optimized CSR modules tailored to user-defined specifications. This approach would maintain functional equivalence and QoR benefits with the manually optimized designs, while enhancing usability and reducing manual effort. Such a methodology could enable a hybrid design flow, where the CSR microarchitecture is generated with high optimization and resource efficiency, while the remaining RISC-V processor is implemented behaviorally for ease of modification. This balance would further facilitate tradeoffs among performance, resource utilization, power consumption, and design productivity, thereby improving the overall practicality and adoption potential of the proposed CSR implementation.

6 Conclusions and Future Works

Softcore processors offer a flexible and extensible foundation to implement specialized computing platforms on FPGAs. While prevailing design efforts gravitate toward datapath computational throughput, effective processor operation also demands robust and efficient mechanisms for privileged control. In RISC-V, CSRs embody this management layer, enabling capabilities such as exception handling, system configuration, debugging, and performance monitoring. Yet, large-scale deployment of CSRs in FPGA fabrics introduce substantial hurdles, the most notable of which is the requirement for decoding and multiplexing their deep register space while maintaining parallel access to control semantics. As studied, conventional LUT-based solutions can result in considerable resource consumption and degraded timing performance.

This work presents a refined approach to CSR subsystem design, informed by the structure and constraints of FPGA platforms. To this end, a minimal microarchitectural environment was constructed, allowing isolation of the CSR handling logic for systematic evaluation of alternative access strategies. Initially, multiple optimizations of conventional behavioral implementations were examined; however, despite measurable improvements, the primary performance bottleneck remained. Consequently, a different design paradigm was adopted, introducing a microarchitectural strategy that partitions responsibilities across heterogeneous FPGA resources to overcome these limitations. Specifically, in the culminating design, decoding is delegated to BRAMs, wide multiplexing is allocated to DSP blocks, and routing is enhanced through judicious optimization of LUT, CARRY4 and FF structures. The proposed enhancements were systematically evaluated in terms of performance, area, and dynamic power consumption, demonstrating notable improvement. As discussed, the performance gains ranged from 50% to over 300%, reaching a maximum of 250 MHz in the leading implementation, whilst simultaneously reducing area and power utilization.

Despite demonstrating substantial QoR enhancement, the proposed microarchitecture entails a discernible tradeoff: increased structural complexity compared to traditional behavioral implementations. The deliberate distribution of logic across heterogeneous FPGA resources, although effective, can result in designs that are more difficult to read, maintain, and adapt. To counteract this, future endeavors will explore the development of a code generation framework capable of automatically synthesizing customized CSR subsystems based on user-defined parameters. Developing such a generator represents a key next step toward transforming this expert-driven approach into a broadly applicable and maintainable design methodology, combining the benefits of hardware-aware optimization with the portability of higher-level abstractions. Moreover, certain architectural techniques explored here, such as BRAM-based one-hot decoding or DSP-assisted multiplexing, may also prove beneficial in unrelated FPGA systems that face similar performance constraints. These design patterns can thus be readily applied in other contexts, illustrating the generalizability of the underlying design principles described in this publication.

Furthermore, the integration of the optimized CSR module into a full-fledged processor pipeline is forthcoming, enabling end-to-end evaluation under authentic workloads and operating conditions. Such an evaluation represents a crucial next step, permitting a twofold assessment of the broader system-level impact arising from the proposed optimizations. First, it will allow analysis of their influence on the critical path of the processor and potential secondary effects emanating from the CSR optimization, as elucidated earlier. Second, future experiments could examine how CSR interactions affect overall pipeline operation efficiency (throughput, stalls, etc.) under different workloads and execution contexts. This will provide a more comprehensive view of the architectural implications, aiding a more nuanced understanding of the speedup achieved by the proposed design in an application-dependent fashion. Moreover, incorporating the optimized CSR subsystem into other additional processors would enable comparison with an external reference, thereby further substantiating the significance of this research.

More fundamentally, this study highlights the enduring relevance of hardware-conscious design practices in the engineering of digital systems. As the field evolves towards a more abstract paradigm, the ability to recognize and consider the physical and architectural constraints that influence implementation outcomes becomes essential. While behavioral modeling and HLS have substantially lowered the barrier to FPGA development, an uncritical reliance on these abstractions risks obscuring crucial architectural considerations essential for efficient hardware utilization. This does not entail a wholesale rejection of abstraction, but rather a call for its judicious application. By grounding design decisions in a concrete understanding of the target substrate, engineers can continue to leverage abstraction as a powerful tool while avoiding the pitfalls of performance opacity and structural inefficiency. In this light, the work presented here serves not only as a practical contribution to CSR microarchitecture design, but also as a broader argument for the reintegration of hardware-awareness into contemporary digital design methodologies, safeguarding the long-term evolution of robust, high-performance, and resource-efficient electronic systems.

Code Availability and Reproducibility

The source code, constraints and build scripts associated with this work have been made publicly available to facilitate reproducibility and future extensions. Two repositories have been established:

Repository A: Reproducibility Release [39]. Dedicated to host the configurations evaluated and reported in this article. This repository contains all the RTL sources, constraint files, and Tcl scripts necessary to reproduce the reported implementation results. In addition, a design checkpoint is provided for each version, which enables implementation of the exact results detailed in this document despite the inherently stochastic nature of FPGA implementation tools.

Repository B: Ongoing Development [40]. Designated to contain the most up-to-date version of the proposed DSPmux microarchitecture, including complete source code and automated build flow.

References

- [1] Jiashu Xu. 2024. Comparison and application of FPGA and ASIC in digital system design. In *Proceedings of the ICDSM '24*. ACM, New York, NY, 216–220. DOI: <https://doi.org/10.1145/3686081.3686117>
- [2] Pablo Valerio. 2025. RISC-V Turns 15 with Fast Global Adoption. *EE Times*. Retrieved June 15, 2025 from <https://web.archive.org/web/202506/https://www.eetimes.com/risc-v-turns-15-with-fast-global-adoption/>
- [3] Shreyas Sharma. 2024. Understanding RISC-V: The Open Standard Instruction Set Architecture. Retrieved June 15, 2025 from <https://web.archive.org/web/202502/https://www.wevolver.com/article/risc-v-instruction-set>
- [4] Roger Boada Gardenyes. 2011. Trends and Patterns in ASIC and FPGA Use in Space Missions and Impact in Technology Roadmaps of ESA. Retrieved June 15, 2025 from <https://web.archive.org/upcommons.upc.edu/handle/2099.1/17994>
- [5] Sebastián Sánchez, Manuel Prieto, Óscar R. Polo, Pablo Parra, Antonio da Silva, Óscar Gutiérrez, Ronald Castillo, Javier Fernández, and Javier Rodríguez-Pacheco. 2013. HW/SW co-design of the instrument control unit for the energetic particle detector on-board solar orbiter. *Advances in Space Research* 52, 6 (2013), 989–1007. DOI: <https://doi.org/10.1016/j.asr.2013.05.029>
- [6] Eduardo Ortega, Alejandro Vicente, Agustín Martínez, Óscar Rodríguez, Manuel Prieto, Pablo Parra, Antonio Da Silva, and Sebastián Sánchez. 2024. Enhancing efficiency in spaceborne phased array systems. *Digital Signal Processing* 155 (2024), 104732. DOI: <https://doi.org/10.1016/j.dsp.2024.104732>
- [7] Borja Losa, Pablo Parra, Antonio Da Silva, Óscar R. Polo, J. Ignacio G. Tejedor, Agustín Martínez, Sebastián Sánchez, and David Guzmán. 2025. Hardware-assisted virtualization extensions for LEON processors in mixed-criticality systems. *Microprocessors and Microsystems* 112 (2025), 105130. DOI: <https://doi.org/10.1016/j.micpro.2024.105130>
- [8] Iván Gamino del Río, Agustín Martínez Hellín, Óscar R. Polo, Miguel Jiménez Arribas, Pablo Parra, Antonio da Silva, Jonatan Sánchez, and Sebastián Sánchez. 2020. A RISC-V processor design for transparent tracing. *Electronics* 9, 11 (2020), 1873. DOI: <https://doi.org/10.3390/electronics9111873>
- [9] Miguel Jiménez Arribas, Agustín Martínez Hellín, Manuel Prieto Mateo, Iván Gamino del Río, Andrea Fernández Gallego, Óscar Rodríguez Polo, Antonio da Silva, Pablo Parra, and Sebastián Sánchez. 2025. Design and implementation of a synchronous hardware performance monitor for a RISC-V space-oriented processor. *Microprocessors and Microsystems* 112 (2025), 105132. DOI: <https://doi.org/10.1016/j.micpro.2024.105132>
- [10] Andrea Fernández Gallego, Miguel Jiménez Arribas, Iván Gamino del Río, Agustín Martínez Hellín, Manuel Prieto Mateo, Óscar Rodríguez Polo, Antonio da Silva, Pablo Parra, and Sebastián Sánchez. 2024. Count overflow and privilege mode filtering extension implementation on a RISC-V on-board processor. *Microprocessors and Microsystems* 109 (2024), 105084. DOI: <https://doi.org/10.1016/j.micpro.2024.105084>
- [11] Digilent. 2018. Nexys A7 Reference Manual. *Tech. Rep. Digilent*. Retrieved June 15, 2025 from <https://web.archive.org/web/20241107031931/https://digilent.com/reference/programmable-logic/nexys-a7/reference-manual>
- [12] Xilinx. 2020. 7 Series FPGAs Data Sheet: Overview (DS180). Retrieved October 2020 from <https://web.archive.org/web/202506/https://docs.amd.com/api/khuv/documents/2LByHkO-nSZXcei2D55fTg/content?>
- [13] Jiri Gaisler. 1998. A structured VHDL design method. *Fault-tolerant Microprocessors for Space Applications*. Retrieved from https://web.archive.org/web/202504/https://download.gaisler.com/research_papers/vhdl2proc.pdf
- [14] Ian Kuon and Jonathan Rose. 2007. Measuring the gap between FPGAs and ASICs. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 26, 2 (2007), 203–215. DOI: <https://doi.org/10.1109/TCAD.2006.884574>
- [15] Jason Cong, Jason Lau, Gai Liu, Stephen Neuendorffer, Peichen Pan, Kees Vissers, and Zhiru Zhang. 2022. FPGA HLS today: Successes, challenges, and opportunities. *ACM Transactions on Reconfigurable Technology and Systems* 15, 4 (2022), 51:1–51:42. DOI: <https://doi.org/10.1145/3530775>
- [16] Sakari Lahti and Timo D. Hämäläinen. 2025. High-level synthesis for FPGAs—A hardware engineer’s perspective. *IEEE Access* 13 (2025), 28574–28593. DOI: <https://doi.org/10.1109/ACCESS.2025.3540320>
- [17] Licheng Guo, Yuze Chi, Jie Wang, Jason Lau, Weikang Qiao, Ecenur Ustun, Zhiru Zhang, and Jason Cong. 2021. AutoBridge: Coupling coarse-grained floorplanning and pipelining for high-frequency HLS design on multi-die FPGAs. In *Proceedings of the 2021 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*. ACM, New York. DOI: <https://doi.org/10.1145/3431920.3439289>
- [18] Andrew Waterman, Yunsup Lee, Rimas Avizienis, David Patterson, and Krste Asanovic. 2025. The RISC-V Instruction Set Manual: Volume II: Privileged Architecture. Tech rep RISC-V International. Retrieved June 25, 2025 from <https://web.archive.org/web/20250609011514/https://riscv.github.io/riscv-isa-manual/snapshot/privileged/>

- [19] Robert Golla and A. Waterman. 2018. Issue #304: Atomic CSR Read-Write Difficult to Implement Efficiently on RISC-V Manual. Retrieved June 16, 2025 from <https://web.archive.org/github.com/riscv/riscv-isa-manual/issues/304>
- [20] Charles Eric Laforest, Zimo Li, Tristan O'rourke, Ming G. Liu, and J. Gregory Steffan. 2014. Composing multi-ported memories on FPGAs. *ACM Transactions on Reconfigurable Technology and Systems* 7, 3 (2014), 16:1–16:23. DOI: <https://doi.org/10.1145/2629629>
- [21] Ameer M. S. Abdelhadi and Guy G. F. Lemieux. 2016. Modular switched multiported SRAM-Based memories. *ACM Transactions on Reconfigurable Technology and Systems* 9, 3 (2016), 22:1–22:26. DOI: <https://doi.org/10.1145/2851506>
- [22] Charles Eric Laforest. 2019. Multi-Ported Memories for FPGAs (and ASICs too). Retrieved October 22, 2025 from <https://web.archive.org/fpgacpu.ca/multiport/index.html>
- [23] Sundar Iyer and Da Chuang. 2012. Algorithmic Memory Brings an Order of Magnitude Performance Increase to Next Generation SoC Memories. Tech rep Memoir Systems. Retrieved from <https://web.archive.org/yuba.stanford.edu/~sundaes/Papers/DesignCon-AlgMem.pdf>
- [24] Ken Chapman. 2008. Multiplexer Selection (WP274 v1.0). Retrieved June 16, 2025 from <https://web.archive.org/web/202506/docs.amd.com/api/khub/documents/Z4IWdT2oFkp8YCG3xNHW2A/content>
- [25] Ken Chapman. 2014. Multiplexer Design Techniques for Datapath Performance with Minimized Routing Resources, v1.2. Retrieved from <https://web.archive.org/docs.amd.com/api/khub/documents/9pOn~3NV8ApAbwglqp6MJQ/content>
- [26] Xilinx. 2018. 7 Series DSP48E1 Slice User Guide (UG479). Retrieved July 2, 2025 from https://web.archive.org/docs.amd.com/api/khub/documents/gu4oRPFEh_Pm2uaAlfY6Kg/content#G6.278063
- [27] Xilinx. 2022. Vivado Design Suite User Guide: Design Analysis and Closure Techniques. Retrieved May 2022 from https://web.archive.org/xilinx.com/support/documents/sw_manuals/xilinx2022_1/ug906-vivado-design-analysis.pdf
- [28] Xilinx. 2021. UltraScale Architecture DSP Slice User Guide. Retrieved October 11, 2025 from <https://web.archive.org/docs.amd.com/api/khub/documents/pTysoma4TYgNH95BrY1Sbw/content?#G3.371980>
- [29] Xilinx. 2022. Versal DSP Engine Architecture Manual (AM004). Retrieved October 11, 2025 from https://web.archive.org/docs.amd.com/api/khub/maps/i4DTfkjq8~7RhoNZm1tsQ/attachments/TvxJUW~wcS_iqA0VEejmuQ-i4DTfkjq8~7RhoNZm1tsQ/content#_OPENTOPIC_TOC_PROCESSING_d114e1415
- [30] Xilinx. 2021. UltraScale Architecture Memory Resources User Guide. Retrieved from <https://web.archive.org/docs.amd.com/api/khub/documents/7~v2GDujU~qYXF1VUBLMag/content#G3.377066>
- [31] Xilinx. 2020. Versal ACAP Memory Resources Architecture Manual. Retrieved from https://web.archive.org/docs.amd.com/api/khub/maps/wWM~XscHykclBs15LPC6yw/attachments/4li_Ng5IVAQl~SMchfeVPQ-wWM~XscHykclBs15LPC6yw/content#_OPENTOPIC_TOC_PROCESSING_d103e3984
- [32] Intel. 2021. Intel Memory Block Types. Retrieved October 11, 2025 from <https://web.archive.org/www.intel.com/content/www/us/en/docs/programmable/683240/17-0/memory-block-types.html>
- [33] Intel. 2017. M20K Memory Block Definition. Retrieved October 11, 2025 from https://web.archive.org/www.intel.com/content/www/us/en/programmable/quartushelp/17.0/reference/glossary/def_m20k.htm
- [34] Intel. 2017. M144K Memory Block Definition. Retrieved October 11, 2025 from https://web.archive.org/www.intel.com/content/www/us/en/programmable/quartushelp/17.0/reference/glossary/def_m144.htm
- [35] Intel. 2019. Intel Stratix 10 Variable Precision DSP Blocks User Guide Document. Retrieved October 11, 2025 from https://web.archive.org/cdrdv2-public.intel.com/705269/ug-s10-dsp-19-3-683832-705269.pdf#_OPENTOPIC_TOC_PROCESSING_d116e9459
- [36] Intel. 2017. Cyclone 10 GX Native Fixed Point DSP IP Core Signals. Retrieved October 11, 2025 from. Retrieved from <https://web.archive.org/www.intel.com/content/www/us/en/docs/programmable/683739/current/ip-core-signals.html>
- [37] Xilinx. 2022. Vivado User Guide, UG904 (v2022.2). Retrieved from https://web.archive.org/xilinx.com/support/documents/sw_manuals/xilinx2022_2/ug904-vivado-implementation.pdf#_OPENTOPIC_TOC_PROCESSING_d114e10546
- [38] Yehya Nasser, Jean-Christophe Prévotet, Maryline Héland, and J. Lorandel. 2017. Dynamic power estimation based on switching activity propagation. In *Proceedings of the 27th FPL Conference*. DOI: <https://doi.org/10.23919/FPL.2017.8056783>
- [39] Miguel Jiménez Arribas. 2025. PerfOptCSRs Results Repository. SRG. Retrieved October 24, 2025 from https://web.archive.org/web/202601/https://github.com/uah-srg-tech/PerfOptCSRs_Results
- [40] Miguel Jiménez Arribas. 2025. PerfOptCSRs Repository: A Microarchitecture for Optimized RISC-V CSR Access. SRG. Retrieved October 24, 2025 from <https://web.archive.org/web/2026/github.com/uah-srg-tech/PerfOptCSRs>

Received 23 August 2025; revised 4 November 2025; accepted 22 December 2025